Using R for Statistical Analysis

Basic Level

Michele Scandola

2026-11-03

Indice

1 About myself **4**

2 Learning objectives **5**

3 About R **5**
3.1 Why Use R as a PhD Student? . . . 5
3.2 How to donwload R and its interface . . . 5
3.3 What R Studio is adding to R? . . . 5
3.4 How to download R Studio and its inteface . . . 6
3.5 What is the `tidyverse`? . . . 6
3.6 How to install `tidyverse`? . . . 6
3.7 Simple difference between base R and tidyverse . . . 7

4 The R language for statistical computing **8**
4.1 . . . 8
4.2 R scripts . . . 8

5 The R Syntax **9**
5.1 Variables . . . 9
5.2 . . . 9
5.3 . . . 9
5.4 . . . 10
5.5 . . . 10
5.6 . . . 10
5.7 . . . 10
5.8 . . . 11
5.9 . . . 11
5.10 . . . 11
5.11 . . . 12
5.12 . . . 13
5.13 . . . 13
5.14 . . . 13
5.15 . . . 13
5.16 . . . 14
5.17 . . . 14
5.18 . . . 15
5.19 . . . 15
5.20 Dataframe . . . 15

5.21 Tibble . . . 15
5.22 . . . 16
5.23 Single row . . . 16
5.24 Set of rows - Base R . . . 16
5.25 Set of rows - `tidyverse` 1 . . . 16
5.26 Set of rows - `tidyverse` 2 . . . 17
5.27 Practice! . . . 17
5.28 . . . 17
5.29 . . . 17
5.30 . . . 18
5.31 . . . 18
5.32 . . . 19
5.33 . . . 19
5.34 . . . 19
5.35 . . . 20
5.36 Operators . . . 20
5.37 . . . 20
5.38 . . . 20
5.39 . . . 21
5.40 . . . 21
5.41 Functions . . . 21
5.42 . . . 21
5.43 . . . 22
5.44 . . . 22
5.45 . . . 22
5.46 . . . 22
5.47 . . . 23
5.48 . . . 23
5.49 . . . 23
5.50 Practice! . . . 23

6 R Libraries **24**
6.1 What are Libraries in R? . . . 24
6.2 Why Use Libraries? . . . 24
6.3 Installing Libraries . . . 24
6.3.1 Syntax: . . . 24
6.3.2 Example: . . . 24
6.4 Loading Libraries . . . 24
6.4.1 Syntax: . . . 24
6.4.2 Example: . . . 24
6.5 Checking Installed Libraries . . . 24
6.5.1 Example: . . . 25
6.6 Commonly Used Libraries in R . . . 25
6.7 Updating Libraries . . . 25
6.7.1 Syntax: . . . 25
6.7.2 Example: . . . 25
6.8 Unloading Libraries . . . 25
6.8.1 Syntax: . . . 25
6.8.2 Example: . . . 25

7 Writing R scripts **25**
7.1 How to load a dataset . . . 25

7.2 CSV format . . . 25
7.3 XLSX format . . . 26
7.4 Other formats . . . 26
7.5 How to explore a dataset . . . 26
7.6 How to clean a dataset . . . 28
7.7 Practice! . . . 29
7.8 Exploratory Data Analysis . . . 29
7.9 Checking for outliers . . . 33
7.10 Statistical Analysis . . . 35
7.11 Data Visualization . . . 37
7.12 Combine all elements into a unique script - Base R . . . 40
7.13 Combine all elements into a unique script - `tidyverse` . . . 47

8 Focus on statistical tests **56**
8.1 Correlation . . . 56
8.1.1 What It Measures . . . 56
8.1.2 Real Dataset Example: `mtcars` . . . 57
8.1.3 Assumptions . . . 57
8.1.4 Visualization . . . 57
8.2 T-Tests . . . 58
8.2.1 Types of T-Tests . . . 58
8.2.2 Real Dataset Example: `ToothGrowth` . . . 58
8.2.3 Assumptions . . . 59
8.2.4 Check Assumptions . . . 59
8.2.5 Visualization . . . 60
8.3 Wilcoxon Test . . . 62
8.3.1 When to Use . . . 62
8.3.2 Real Dataset Example: `ToothGrowth` . . . 63
8.3.3 Assumptions . . . 63
8.4 Linear Models (LM) . . . 63
8.4.1 What They Do . . . 63
8.4.2 Real Dataset Example: `mtcars` . . . 63
8.4.3 Assumptions . . . 64
8.4.4 Diagnostic Plots . . . 64
8.5 Model Inference . . . 64
8.5.1 Coefficients Interpretation . . . 64
8.5.2 Confidence Intervals . . . 65
8.6 Visualization: Linear Model . . . 65
8.7 Summary Table . . . 67

9 RStudio Projects and the R Workspace **67**
9.1 What is an RStudio Project? . . . 67
9.2 Creating an RStudio Project . . . 67
9.3 Benefits of Using Projects . . . 67
9.4 R Workspace: What's in It? . . . 68
9.5 Workspace Files . . . 68
9.6 Best Practices . . . 68
9.7 Summary . . . 68

10 Summary **68**
10.1 Thank you for your attention! . . . 69

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.1.6
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.2     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becc
Caricamento del pacchetto richiesto: lme4

Caricamento del pacchetto richiesto: Matrix


Caricamento pacchetto: 'Matrix'


I seguenti oggetti sono mascherati da 'package:tidyr':

    expand, pack, unpack


************
Welcome to afex. For support visit: http://afex.singmann.science/

- Functions for ANOVAs: aov_car(), aov_ez(), and aov_4()
- Methods for calculating p-values with mixed(): 'S', 'KR', 'LRT', and 'PB'
- 'afex_aov' and 'mixed' objects can be passed to emmeans() for follow-up tests
- Get and set global package options with: afex_options()
- Set sum-to-zero contrasts globally: set_sum_contrasts()
- For example analyses see: browseVignettes("afex")
************


Caricamento pacchetto: 'afex'


Il seguente oggetto è mascherato da 'package:lme4':

    lmer
```

1 About myself

- *Position:* Associate Professor @ Dept. of Human Sciences, UniVR
- *Academic Sector:* PSIC-01/B – Neuropsychology and Cognitive Neuroscience
- *Research Interests:*
  - Body, action, space, and time representations and their influence on cognition.
  - The influence of social disconnection on cognition.
  - Metascience and methodology.

2 Learning objectives

By the end of this session, you will be able to:

- *Write and execute basic R scripts.*
- *Import and manipulate datasets using R.*
- *Perform fundamental statistical analyses.*
- *Visualize data effectively with base R plots.*

3 About R

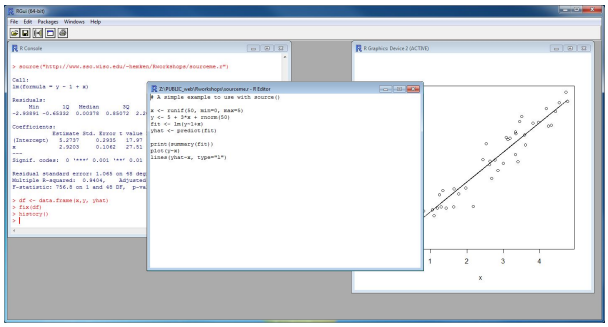
3.1 Why Use R as a PhD Student?

- Free & Open-Source
- Interoperable
- Powerful for Data Analysis
- Reproducibility & Transparency
- Extensive Community & Support
- Integration with Other Tools
- Visualization & Reporting
- Customizability
- Automation
- Strong Academic Adoption
- Version Control & Collaboration

3.2 How to donwload R and its interface

Website: https://www.r-project.org/

The default R GUI on Windows:



The default R Interfaces on Mac and Linux is the Terminal.

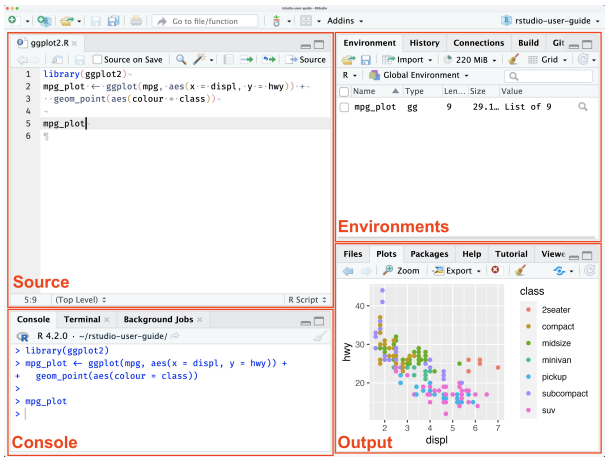
3.3 What R Studio is adding to R?

- User-Friendly Interface
- Script Management
- Data & Variable View
- Built-in Debugging Tools
- Powerful Visualization Support
- R Markdown & Report Generation

- Visualization & Reporting
- Package Management
- Project Organization
- Shiny App Development

3.4 How to download R Studio and its inteface

Website: https://posit.co/products/open-source/rstudio/



3.5 What is the `tidyverse`?

The **tidyverse** is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures.

. . .

Key Features of the `tidyverse`

- Consistent Grammar
- Pipes (`%>%` or `|>`): Enables chaining operations for clean and efficient code.
- Tibble Data Frames: More user-friendly than base R's data.frame.

3.6 How to install `tidyverse`?

```
install.packages("tidyverse")
```

You can find an online book here: https://r4ds.hadley.nz/



3.7 Simple difference between base R and tidyverse

Tabella 1: Mean and standard deviations of simulated data

group	value
A	4.953 (0.981)
B	6.178 (0.835)

. . .

In base R:

```
# Perform independent t-test
t_test_result <- t.test(group_A, group_B, var.equal = TRUE)
## this is equivalen:
## t_test_result <- t.test(value ~ group, var.equal = TRUE, data = DATA)

# Print results
print(t_test_result)
```

```
    Two Sample t-test

data:  group_A and group_B
t = -5.2098, df = 58, p-value = 2.616e-06
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -1.6962870 -0.7545972
sample estimates:
mean of x mean of y
 4.952896  6.178338
```

. . .

In `tidyverse`

```r
# Perform t-test using broom package
library(broom)

t_test_result <- DATA %>%
  t.test(value ~ group, data = .) %>%
  tidy()

# Print results
print(t_test_result)
```

```
# A tibble: 1 x 10
  estimate estimate1 estimate2 statistic    p.value parameter conf.low conf.high
     <dbl>     <dbl>     <dbl>     <dbl>      <dbl>     <dbl>    <dbl>     <dbl>
1    -1.23      4.95      6.18     -5.21 0.00000275      56.6    -1.70    -
0.754
# i 2 more variables: method <chr>, alternative <chr>
```

4 The R language for statistical computing

4.1

R is a programming language specifically designed for data analysis and graphics.

- Dialect of S
- 1988 - S2: RA Becker, JM Chambers, A Wilks
- 1992 - S3: JM Chambers, TJ Hastie
- 1998 - S4: JM Chambers
- R: initially written by Ross Ihaka and Robert Gentleman at Dep. of Statistics of University of Auckland, New Zealand during 1990s.
- Since 1997: international *R-core* team of 15 people with access to common CVS archive.

. . .

```r
R.version.string
```

```
[1] "R version 4.5.2 (2025-10-31 ucrt)"
```

4.2 R scripts

R scripts are textual files saved with the `.R` extension.

. . .

This is a typical R script

```r
## initial comments explaining what is doing the script
## Author, Start Date - Last modification Date
## version

rm( list = ls() ) ## needed to "clean" the workspace's memory
graphics.off()    ## needed to "clean" the graphics' memory

library(tidyverse)## for tidyverse functions
library(afex)     ## for linear models
library(ggplot2)  ## for advanced graphics
```

```r

DATA <-           ## loading the dataset
  read.csv(
    "dataset.csv"
  )

summary(DATA)     ## a summary of the main characteristics of the dataset
str(DATA)         ## the structure of the dataset

## fitting the linear model
mdl <- lm(value ~ group, data = DATA)

anova( mdl )      ## hypothesis testing

## plotting the results
boxplot( value ~ group, data = DATA )
```

5 The R Syntax

5.1 Variables

- named storage location
- the value is set by the user
- the value can be changed after being initially set

. . .

In naming variables you can use alphanumeric characters, dots (".") and underscores ("_").

You cannot use a number as first character.

5.2

EXAMPLES:

- `avgRT` OK
- `_avrRT` OK
- `.avgRT` OK
- `avgRT3` OK

- `avg RT` RED
- `#avrRT` RED
- `avg-RT` RED
- `3avgRT` RED

. . .

It is case sensitive, so `avgRT` $\neq$ `avgrt`

5.3

To assign a value to a variable, you can use

```r
avgRT = 300

avgRT <- 300
```

```
4
5 300 -> avgRT
```

. . .

```
avgRT = 300
avgRT
```

```
[1] 300
```

```
avgRT <- 300
avgRT
```

```
[1] 300
```

```
300 -> avgRT
avgRT
```

```
[1] 300
```

5.4

Different typologies of variables:

```
my_var <- 36
class(my_var)
```

```
[1] "numeric"
```

5.5

Different typologies of variables:

```
my_var <- "hello"
class(my_var)
```

```
[1] "character"
```

5.6

Different typologies of variables:

```
my_var <- TRUE
class(my_var)
```

```
[1] "logical"
```

5.7

Different typologies of variables:

```
my_var <- c(36, 74, 23, 34)
class(my_var)
```

```
[1] "numeric"
```

```
my_var[2] + my_var[3]
```

```
[1] 97
```

5.8

Different typologies of variables:

```
my_var <- c("M", 74, 23, 34)
class(my_var)
```

```
[1] "character"
```

```
my_var[2] + my_var[3]
```

```
Error in `my_var[2] + my_var[3]`:
! argomento non numerico passato ad un operatore binario
```

5.9

Different typologies of variables:

```
my_var <- factor(c("M", "F", "F"))
class(my_var)
```

```
[1] "factor"
```

```
my_var[2] + my_var[3]
```

```
Warning in Ops.factor(my_var[2], my_var[3]): '+' è senza senso per variabili
factor
```

```
[1] NA
```

5.10

Different typologies of variables:

```
my_var <- cbind(
    c(1, 2, 3, 4),
    5:8,
    c(9, 10:12)
  )

class(my_var)
```

```
[1] "matrix" "array"
```

. . .

```
my_var2 <- matrix(
  1:12,
  ncol = 3
)

class(my_var2)
```

```
[1] "matrix" "array"
```

. . .

```
my_var
```

```
     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

```
my_var2
```

```
     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

. . .

```
my_var[2,3]
```

```
[1] 10
```

```
my_var[2,]
```

```
[1]  2  6 10
```

. . .

```
my_var3 <- matrix(
  nrow = 4,
  ncol = 3
)

class(my_var3)
```

```
[1] "matrix" "array"
```

```
my_var3
```

```
     [,1] [,2] [,3]
[1,]   NA   NA   NA
[2,]   NA   NA   NA
[3,]   NA   NA   NA
[4,]   NA   NA   NA
```

5.11

Different typologies of variables:

```
my_var <- data.frame(
  age = c(27, 34, 19),
  sex = c("M", "F", "F"),
  RT = c(300, 123, 678)
)
class(my_var)
```

```
[1] "data.frame"
```

5.12

Different typologies of variables:

```
my_var <- tibble(
  age = c(27, 34, 19),
  sex = c("M", "F", "F"),
  RT = c(300, 123, 678)
)
class(my_var)
```

```
[1] "tbl_df"     "tbl"        "data.frame"
```

5.13

Vectors

```
my_vector <- c(1, 2, 3, 4, 5)
class(my_vector)
```

```
[1] "numeric"
```

```
length(my_vector)
```

```
[1] 5
```

```
dim(my_vector)
```

```
NULL
```

5.14

Vectors

```
my_vector <- 1:5
class(my_vector)
```

```
[1] "integer"
```

```
length(my_vector)
```

```
[1] 5
```

```
dim(my_vector)
```

```
NULL
```

5.15

Vectors

```
my_vector <- cbind(1, 2, 3, 4, 5)
class(my_vector)
```

```
[1] "matrix" "array"
```

```
length(my_vector)
```

```
[1] 5
```

```r
dim(my_vector)
```

```
[1] 1 5
```

5.16

Vectors

We can select a subset from a vector

```r
my_vector[1:3]
```

```
[1] 1 2 3
```

. . .

We can exclude a value from a vector

```r
my_vector[-1]
```

```
[1] 2 3 4 5
```

. . .

We can select a non-contiguous subset from a vector

```r
my_vector[c(2,4)]
```

```
[1] 2 4
```

. . .

We can exclude non-contiguous values from a vector

```r
my_vector[-c(2,4)]
```

```
[1] 1 3 5
```

. . .

However, if we do not assign the new value to the vector, it does not change.

```r
my_vector
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    2    3    4    5
```

5.17

Vectors

We can modify a specific element of a vector

```r
my_vector[2] <- my_vector[1] + my_vector[5]
my_vector
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    6    3    4    5
```

. . .

We can remove a specific element of a vector

```r
my_vector <- my_vector[-3]
my_vector
```

```
[1] 1 6 4 5
```

5.18

Data frame or tibbles

```r
## generally, you obtain them by reading some external database
set.seed(1)
my_df <- data.frame(
  age = round( rnorm(10, mean = 50, sd = 23) ),
  sex = factor(c("M", "M", "M", "M", "M", "F", "F", "F", "F", "F")),
  RT = rnorm(10, mean = 5, sd = 3) + rexp( 10, rate = 0.1 )
)

set.seed(1)
my_tb <- tibble(
  age = round( rnorm(10, mean = 50, sd = 23) ),
  sex = factor(c("M", "M", "M", "M", "M", "F", "F", "F", "F", "F")),
  RT = rnorm(10, mean = 5, sd = 3) + rexp( 10, rate = 0.1 )
)
```

5.19

Data frame or tibbles

We can print them entirely

5.20 Dataframe

```r
print(my_df, digits = 3)
```

```
   age sex     RT
1   36   M 15.954
2   54   M  9.111
3   31   M  8.795
4   87   M -0.583
5   58   M  8.969
6   31   F 10.652
7   61   F 44.541
8   67   F 19.565
9   63   F 17.432
10  43   F 21.135
```

5.21 Tibble

```r
print(my_tb)
```

```
# A tibble: 10 x 3
     age sex       RT
   <dbl> <fct>  <dbl>
```

```
 1    36 M     16.0
 2    54 M      9.11
 3    31 M      8.79
 4    87 M     -0.583
 5    58 M      8.97
 6    31 F     10.7
 7    61 F     44.5
 8    67 F     19.6
 9    63 F     17.4
10    43 F     21.1
```

5.22

Data frame or tibbles

We can access to a single column of a dataset.

```
my_df$age
```

```
 [1] 36 54 31 87 58 31 61 67 63 43
```

```
my_df[,"age"]
```

```
 [1] 36 54 31 87 58 31 61 67 63 43
```

```
my_df[,1]
```

```
 [1] 36 54 31 87 58 31 61 67 63 43
```

. . .

We can access to a specific row or set of rows of a dataset.

5.23 Single row

```
my_df[2,]
```

```
  age sex       RT
2  54   M 9.110734
```

5.24 Set of rows - Base R

```
my_df[my_df$sex == "M",]
```

```
  age sex         RT
1  36   M 15.9542694
2  54   M  9.1107336
3  31   M  8.7949335
4  87   M -0.5833734
5  58   M  8.9691844
```

5.25 Set of rows - `tidyverse` 1

```
my_df %>%
  filter(sex == "M")
```

```
  age sex         RT
1  36   M 15.9542694
2  54   M  9.1107336
3  31   M  8.7949335
4  87   M -0.5833734
5  58   M  8.9691844
```

5.26 Set of rows - `tidyverse` 2

```
my_tb %>%
  filter(sex == "M")
```

```
# A tibble: 5 x 3
    age sex       RT
  <dbl> <fct>  <dbl>
1    36 M     16.0
2    54 M      9.11
3    31 M      8.79
4    87 M     -0.583
5    58 M      8.97
```

5.27 Practice!

1. Open the file "01-variables.R" and follow the given steps.
2. Once completed, upload your exercise to the Moodle course page.
3. We will hold a session to review and correct the exercise together.

5.28

In R, there are several **special values** that can be assigned to variables.

These values are commonly used in data analysis and programming to handle specific scenarios like missing data, infinite values, or undefined numbers.

Here's a list of important special values in R.

5.29

1. `NA` - Not Available (Missing Values)

- **Description:** Represents **missing** or **undefined** data.
- **Use case:** Used to indicate that a value is missing or unavailable in a dataset.

. . .

Example:

```
x <- NA

is.na(x)
```

```
[1] TRUE
```

```
is.nan(x)
```

```
[1] FALSE
```

```r
is.infinite(x)
```

```
[1] FALSE
```

```r
is.null(x)
```

```
[1] FALSE
```

5.30

2. `NaN` - Not a Number

- **Description:** Represents an **undefined numerical result**, such as 0/0 or infinity minus infinity.
- **Use case:** Used when a mathematical operation returns an undefined result.

. . .

Example:

```r
x <- 0 / 0

is.na(x)
```

```
[1] TRUE
```

```r
is.nan(x)
```

```
[1] TRUE
```

```r
is.infinite(x)
```

```
[1] FALSE
```

```r
is.null(x)
```

```
[1] FALSE
```

5.31

3. `Inf` and `-Inf` - Positive and Negative Infinity

- **Description:** Represents **positive infinity (`Inf`)** and **negative infinity (`-Inf`)**.
- **Use case:** Used when a number exceeds the largest representable finite number in R or when division by zero occurs.

. . .

Example:

```r
x <- 1 / 0  # Inf

is.na(x)
```

```
[1] FALSE
```

```r
is.nan(x)
```

```
[1] FALSE
```

```r
is.infinite(x)
```

```
[1] TRUE
```

```r
is.null(x)
```

```
[1] FALSE
```

5.32

4. `NULL` - Null Object

- **Description:** Represents an **empty** or **non-existent value**.
- **Use case:** Used to represent the absence of any value or object. It's different from `NA` because it means the absence of a value entirely, rather than missing data.

. . .

Example:

```r
x <- NULL

is.na(x)
```

```
logical(0)
```

```r
is.nan(x)
```

```
logical(0)
```

```r
is.infinite(x)
```

```
logical(0)
```

```r
is.null(x)
```

```
[1] TRUE
```

5.33

5. `TRUE` and `FALSE` - Logical Boolean Values

- **Description:** Represents logical **TRUE** and **FALSE** values.
- **Use case:** Used in logical conditions and comparisons.

. . .

Example:

```r
x <- TRUE
y <- FALSE
x & y
```

```
[1] FALSE
```

5.34

6. `Inf` vs `-Inf`

- **`Inf`** is used to represent positive infinity (like `1 / 0`), and **`-Inf`** represents negative infinity (like `-1 / 0`).

. . .

Example:

```
x <- 1 / 0  # Inf

is.na(x)
```

```
[1] FALSE
```

```
is.nan(x)
```

```
[1] FALSE
```

```
is.infinite(x)
```

```
[1] TRUE
```

```
is.null(x)
```

```
[1] FALSE
```

5.35

Summary Table

Value | Description | Use Case Example |

||-|–| | `NA` | Missing or undefined data | `x <- NA` | | `NaN` | Not a number (e.g., 0/0) | `y <- 0 / 0` | | `Inf` | Positive infinity | `positive_inf <- 1 / 0` | | `-Inf` | Negative infinity | `negative_inf <- -1 / 0` | | `NULL` | Represents an empty object or value | `z <- NULL` | | `TRUE` | Logical TRUE | `x <- TRUE` | | `FALSE` | Logical FALSE | `y <- FALSE` |

5.36 Operators

R has several types of operators for performing calculations and data manipulations:

1. Arithmetic Operators (Perform mathematical operations)

Operator | Description | Example (`x <- 10`, `y <- 3`) | Result |

|-||-|–| | `+` | Addition | `x + y` | `13` | | `-` | Subtraction | `x - y` | `7` | | `*` | Multiplication | `x * y` | `30` | | `/` | Division | `x / y` | `3.33` | | `^` or `**` | Exponentiation | `x ^ y` or `x ** y` | `1000` | | `%%` | Modulus (remainder) | `x %% y` | `1` | | `%/%` | Integer Division | `x %/% y` | `3` |

5.37

2. Relational (Comparison) Operators (Compare values, return `TRUE` or `FALSE`)

Operator | Description | Example (`x <- 10, y <- 3`) | Result |

|-|–|–|| | `==` | Equal to | `x == y` | `FALSE` | | `!=` | Not equal to | `x != y` | `TRUE` | | `>` | Greater than | `x > y` | `TRUE` | | `<` | Less than | `x < y` | `FALSE` | | `>=` | Greater than or equal to | `x >= y` | `TRUE` | | `<=` | Less than or equal to | `x <= y` | `FALSE` |

5.38

3. Logical Operators (Used for Boolean logic)

Operator	Description	Example (`x <- TRUE, y <- FALSE`)	Result
`&`	AND (element-wise)	`x & y`	`FALSE`
`&&`	AND (first element only)	`x && y`	`FALSE`
`|`	OR (element-wise)	`x | y`	`TRUE`
`||`	OR (first element only)	`x || y`	`TRUE`
`!`	NOT	`!x`	`FALSE`

5.39

4. Assignment Operators (Used to assign values)

Operator | Description | Example |

|-|–|| | `<-` | Left assignment | `x <- 10` | | `->` | Right assignment | `10 -> x` | | `<<-` | Global assignment | `x <<- 20` | | `=` | Assignment (less common) | `x = 5` |

5.40

5. Special Operators (Used for specific operations)

Operator	Description	Example
`%in%`	Checks if value is in a vector	`3 %in% c(1, 2, 3, 4)` → `TRUE`
`%*%`	Matrix multiplication	`A %*% B`
`%/%`	Integer division	`10 %/% 3` → `3`
`%%`	Modulos division (remainder)	`10 %% 3` → `1`

5.41 Functions

- self-contained block of code performing a specific task
- help organise the code
- make it reusable
- improve readability

. . .

```
## computing the average value without using functions

x <- rnorm(10, mean = 3)

(x[1] + x[2] + x[3] + x[4] + x[5] + x[6] + x[7] + x[8] + x[9] + x[10]) / 10
```

```
[1] 3.186496
```

. . .

```
## computing the average value using functions

mean(x)
```

```
[1] 3.186496
```

5.42

1. Mathematical Functions

Function | Description | Example | Output |

|–|||| | `abs(x)` | Absolute value | `abs(-5)` | `5` | | `sqrt(x)` | Square root | `sqrt(16)` | `4` | | `log(x, base)` | Logarithm (default base `e`) | `log(8, 2)` | `3` | | `exp(x)` | Exponential (`e^x`) | `exp(2)` | `7.389` | | `round(x, digits)` | Rounds a number | `round(3.14159, 2)` | `3.14` | | `ceiling(x)` | Rounds up | `ceiling(3.4)` | `4` | | `floor(x)` | Rounds down | `floor(3.9)` | `3` | | `trunc(x)` | Truncates decimals | `trunc(3.9)` | `3` | | `factorial(x)` | Factorial | `factorial(5)` | `120` |

5.43

2. Statistical Functions

Function | Description | Example | Output |

|||-|–| | `mean(x)` | Arithmetic mean | `mean(c(1,2,3,4))` | `2.5` | | `median(x)` | Median | `median(c(1,2,3,4,5))` | `3` | | `sd(x)` | Standard deviation | `sd(c(1,2,3,4,5))` | `1.58` | | `var(x)` | Variance | `var(c(1,2,3,4,5))` | `2.5` | | `sum(x)` | Sum of elements | `sum(c(1,2,3))` | `6` | | `prod(x)` | Product of elements | `prod(c(1,2,3,4))` | `24` | | `range(x)` | Min and max values | `range(c(2,4,6,8))` | `2 8` | | `quantile(x, p)` | Percentiles | `quantile(c(1,2,3,4,5), 0.5)` | `3` |

5.44

3. Logical and Comparison Functions

Function | Description | Example | Output |

|||–|–| | `all(x)` | Checks if all values are `TRUE` | `all(c(TRUE, TRUE))` | `TRUE` | | `any(x)` | Checks if any value is `TRUE` | `any(c(FALSE, TRUE))` | `TRUE` | | `is.na(x)` | Checks for `NA` values | `is.na(c(1, NA, 3))` | `FALSE TRUE FALSE` | | `which(x)` | Returns indices of `TRUE` values | `which(c(FALSE, TRUE, TRUE))` | `2 3` |

5.45

4. Character/String Functions

Function | Description | Example | Output |

|||–|–| | `nchar(x)` | Number of characters | `nchar("hello")` | `5` | | `toupper(x)` | Convert to uppercase | `toupper("hello")` | `"HELLO"` | | `tolower(x)` | Convert to lowercase | `tolower("HELLO")` | `"hello"` | | `substr(x, start, stop)` | Extract substring | `substr("Hello", 2, 4)` | `"ell"` | | `paste(x, y, sep)` | Concatenate strings | `paste("Hello", "World", sep=" ")` | `"Hello World"` |

5.46

5. Data Manipulation Functions

Function | Description | Example | Output |

|||–|–| | `length(x)` | Number of elements | `length(c(1,2,3))` | `3` | | `sort(x)` | Sort elements | `sort(c(3,1,2))` | `1 2 3` | | `rev(x)` | Reverse elements | `rev(c(1,2,3))` | `3 2 1` | | `unique(x)` | Remove duplicates | `unique(c(1,2,2,3))` | `1 2 3` | | `table(x)` | Frequency count | `table(c("a", "b", "a"))` | `"a" 2, "b" 1` |

5.47

6. Control and Utility Functions

Function | Description | Example | Output |

|||–|–| | `seq(from, to, by)` | Generate a sequence | `seq(1, 10, 2)` | `1 3 5 7 9` | | `rep(x, times)` | Repeat values | `rep(1:3, 2)` | `1 2 3 1 2 3` | | `sample(x, size)` | Random sample | `sample(1:10, 5)` | Varies | | `ifelse(condition, yes, no)` | Conditional | `ifelse(3 > 2, "Yes", "No")` | `"Yes"` |

5.48

7. `tidyverse` Data Manipulation Functions (`dplyr`)

Function | Description | Example |

|–|-|| | `filter(.data, condition)` | Subset rows based on conditions | `filter(df, age > 30)` | | `select(.data, columns)` | Choose specific columns | `select(df, name, age)` | | `mutate(.data, new_col = expression)` | Create or modify columns | `mutate(df, age_10 = age + 10)` | | `arrange(.data, column)` | Sort rows by column | `arrange(df, desc(salary))` | | `group_by(.data, column)` | Group data by a column | `group_by(df, department)` | | `summarize(.data, new_col = summary_func(column))` | Compute summary statistics | `summarize(df, avg_salary = mean(salary))` | | `rename(.data, new_name = old_name)` | Rename columns | `rename(df, emp_name = name)` | | `distinct(.data, column)` | Remove duplicate rows | `distinct(df, city)` | | `count(.data, column)` | Count occurrences | `count(df, gender)` |

5.49

8. other important functions:

Function	Description	Example Usage
`ls()`	List objects in the current environment	`ls()`
`setwd()`	Set the working directory	`setwd("C:/path/to/directory")`
`getwd()`	Get the current working directory	`getwd()`
`save()`	Save objects to a file	`save(x, file = "x_data.RData")`
`save.image()`	Save all objects in the environment to a file	`save.image("my_workspace.RData")`
`help()`	Get help on a function or topic	`help(ls)`
`history()`	View the command history of the current session	`history()`
`savehistory()`	Save the command history to a file	`savehistory("my_history.Rhistory")`
`loadhistory()`	Load command history from a file	`loadhistory("my_history.Rhistory")`

5.50 Practice!

1. Open the file "02-functions.R" and follow the given steps.

2. Once completed, upload your exercise to the Moodle course page.
3. We will hold a session to review and correct the exercise together.

6 R Libraries

6.1 What are Libraries in R?

- **Libraries** (also called **packages**) in R are collections of pre-written functions, data sets, and documentation.
- They extend R's functionality and provide tools for various tasks like data analysis, visualization, statistics, and machine learning.
- You can install and load libraries to access these tools.

6.2 Why Use Libraries?

- **Efficiency**: Libraries save you time by providing pre-built functions.
- **Specialization**: Libraries are tailored to specific tasks (e.g., `ggplot2` or `lattice` for visualization, `dplyr` or `reshape2` for data manipulation).
- **Community Contributions (bright side)**: Thousands of libraries are available, often developed by the R community.
- **Community Contributions (dark side)**: No professional software development. Many libraries can be discontinued.

6.3 Installing Libraries

To use a library, it needs to be installed first.

6.3.1 Syntax:

```
install.packages("library_name")
```

6.3.2 Example:

```
install.packages("ggplot2")
```

6.4 Loading Libraries

Once installed, you need to load a library to access its functions.

6.4.1 Syntax:

```
library(library_name)
```

6.4.2 Example:

```
library(ggplot2)
```

6.5 Checking Installed Libraries

You can check which libraries are installed using the `installed.packages()` function.

6.5.1 Example:

```
installed.packages()
```

6.6 Commonly Used Libraries in R

- `ggplot2`
  - **Purpose**: Data visualization
  - **Example**: Create sophisticated plots and charts.
- `afex`
  - **Purpose**: Analysis of Factorial Experiments.
  - **Example**: Convenience functions for analyzing factorial experiments using ANOVA or mixed models.
- `tidyr` and `reshape2`
  - **Purpose**: Reshaping data
  - **Example**: Pivot data between wide and long formats.
- `shiny`
  - **Purpose**: Build interactive web applications
  - **Example**: Develop dashboards and interactive plots.

6.7 Updating Libraries

Libraries can be updated to the latest version using the `install.packages()` function.

6.7.1 Syntax:

```
install.packages("library_name", repos = "https://cloud.r-project.org/")
```

6.7.2 Example:

```
install.packages("ggplot2", repos = "https://cloud.r-project.org/")
```

6.8 Unloading Libraries

To unload a library during your R session, use the `detach()` function.

6.8.1 Syntax:

```
detach("package:library_name", unload = TRUE)
```

6.8.2 Example:

```
detach("package:ggplot2", unload = TRUE)
```

7 Writing R scripts

7.1 How to load a dataset

7.2 CSV format

```r
DATA <- read.csv(file = "data/data_sleepstudy.csv")
# DATA <- read.csv2(file = "data/data_sleepstudy.csv")
```

7.3 XLSX format

```r
library(readxl)

DATA <- read_xlsx(path = "data/data_sleepstudy.xlsx", sheet = "Sheet 1")
```

7.4 Other formats

```r
lsf.str("package:foreign", pattern = "read")
```

```
read.arff : function (file)
read.dbf : function (file, as.is = FALSE)
read.dta : function (file, convert.dates = TRUE, convert.factors = TRUE, missing.type = FALS
    convert.underscore = FALSE, warn.missing.labels = TRUE)
read.epiinfo : function (file, read.deleted = FALSE, guess.broken.dates = FALSE, thisyear =
    lower.case.names = FALSE)
read.mtp : function (file)
read.octave : function (file)
read.S : function (file)
read.spss : function (file, use.value.labels = TRUE, to.data.frame = FALSE, max.value.labels
    trim.factor.names = FALSE, trim_values = TRUE, reencode = NA, use.missings = to.data.fra
    sub = ".", add.undeclared.levels = c("sort", "append", "no"), duplicated.value.labels =
        "condense"), duplicated.value.labels.infix = "_duplicated_", ...)
read.ssd : function (libname, sectionnames, tmpXport = tempfile(), tmpProgLoc = tempfile(),
    sascmd = "sas")
read.systat : function (file, to.data.frame = TRUE)
read.xport : function (file, ...)
```

7.5 How to explore a dataset

Base R:

```r
# Check the structure of the dataset
str(DATA)
```

```
tibble [180 x 4] (S3: tbl_df/tbl/data.frame)
 $ Reaction: num [1:180] 250 259 251 321 357 ...
 $ Days    : num [1:180] 0 1 2 3 4 5 6 7 8 9 ...
 $ Subject : chr [1:180] "308" "308" "308" "308" ...
 $ Group   : chr [1:180] "A" "A" "A" "A" ...
```

```r
# View the first few rows
head(DATA)
```

```
# A tibble: 6 x 4
  Reaction  Days Subject Group
     <dbl> <dbl> <chr>   <chr>
1     250.     0 308     A
2     259.     1 308     A
```

```
3     251.     2 308     A
4     321.     3 308     A
5     357.     4 308     A
6     415.     5 308     A
```

```
# Check for missing values and quantiles
summary(DATA)
```

```
    Reaction          Days       Subject             Group
 Min.   :194.3   Min.   :0.0   Length:180         Length:180
 1st Qu.:255.4   1st Qu.:2.0   Class :character   Class :character
 Median :288.7   Median :4.5   Mode  :character   Mode  :character
 Mean   :298.5   Mean   :4.5
 3rd Qu.:336.8   3rd Qu.:7.0
 Max.   :466.4   Max.   :9.0
```

```
quantile(DATA$Reaction)
```

```
      0%       25%       50%       75%      100%
194.3322 255.3758 288.6508 336.7521 466.3535
```

```
quantile(DATA$Days)
```

```
  0%  25%  50%  75% 100%
 0.0  2.0  4.5  7.0  9.0
```

. . .

tidyverse:

```
glimpse(DATA)
```

```
Rows: 180
Columns: 4
$ Reaction <dbl> 249.5600, 258.7047, 250.8006, 321.4398, 356.8519, 414.6901, 3~
$ Days     <dbl> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0~
$ Subject  <chr> "308", "308", "308", "308", "308", "308", "308", "308", "308"~
$ Group    <chr> "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "~
```

```
skimr::skim(DATA)
```

Tabella 2: Data summary

Name	DATA
Number of rows	180
Number of columns	4
Column type frequency:	
character	2
numeric	2
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Subject	0	1	3	3	0	18	0
Group	0	1	1	1	0	2	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Reaction	0	1	298.51	56.33	194.33	255.38	288.65	336.75	466.35	
Days	0	1	4.50	2.88	0.00	2.00	4.50	7.00	9.00	

```
DATA %>%
  summarise(across(everything(), ~ sum(is.na(.)), .names = "missing_{.col}"))
```

```
# A tibble: 1 x 4
  missing_Reaction missing_Days missing_Subject missing_Group
             <int>        <int>           <int>         <int>
1                0            0               0             0
```

```
DATA %>%
  summarise(across(where(is.numeric),
                   list(
                     mean = mean,
                     sd = sd,
                     min = min,
                     max = max),
                   na.rm = TRUE)
            )
```

```
Warning: There was 1 warning in `summarise()`.
i In argument: `across(...)`.
Caused by warning:
! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
Supply arguments directly to `.fns` through an anonymous function instead.

  # Previously
  across(a:b, mean, na.rm = TRUE)

  # Now
  across(a:b, \(x) mean(x, na.rm = TRUE))
```

```
# A tibble: 1 x 8
  Reaction_mean Reaction_sd Reaction_min Reaction_max Days_mean Days_sd Days_min
          <dbl>       <dbl>        <dbl>        <dbl>     <dbl>   <dbl>    <dbl>
1          299.        56.3         194.         466.       4.5    2.88        0
# i 1 more variable: Days_max <dbl>
```

7.6 How to clean a dataset

Base R:

```r
# Handling missing values
clean_DATA <- na.omit(DATA)  # Removes rows with NA values
# or, you could impute missing values
DATA$column_name <- ifelse(is.na(DATA$column_name),
     mean(DATA$column_name, na.rm = TRUE), data$column_name)

# Filter data based on certain conditions
flt_DATA <- DATA[DATA$column_name > value,]
```

. . .

tidyverse:

```r
# Handling missing values
clean_DATA <- DATA %>% drop_na()  # Removes rows with NA values
# or, you could impute missing values
DATA <- DATA %>%
  mutate(column_name = replace_na(column_name, mean(column_name, na.rm = TRUE)))

# Filter data based on certain conditions
filtered_data <- data %>% filter(column_name > value)
```

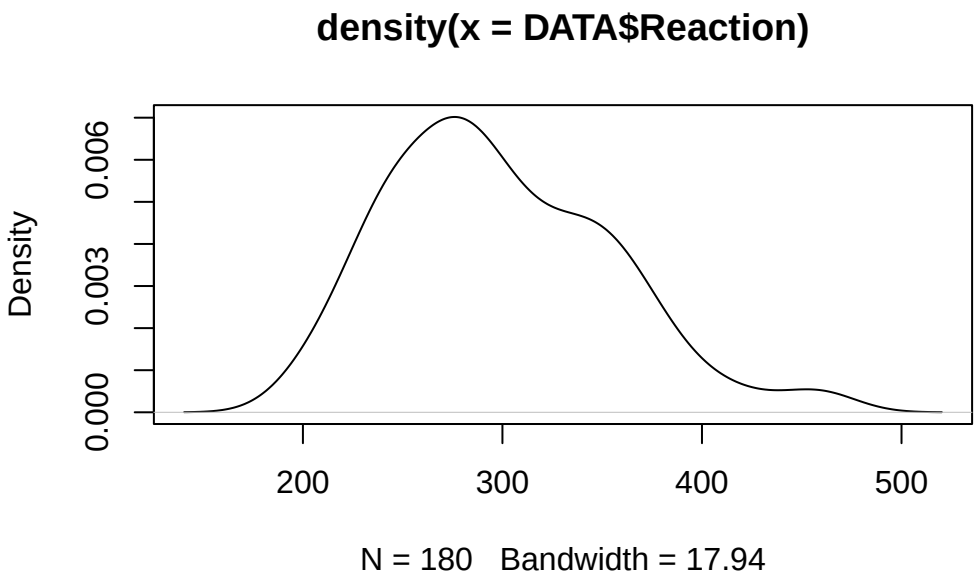
7.7 Practice!

1. Open the file "03-datasets.R" and follow the given steps.
2. Once completed, upload your exercise to the Moodle course page.
3. We will hold a session to review and correct the exercise together.

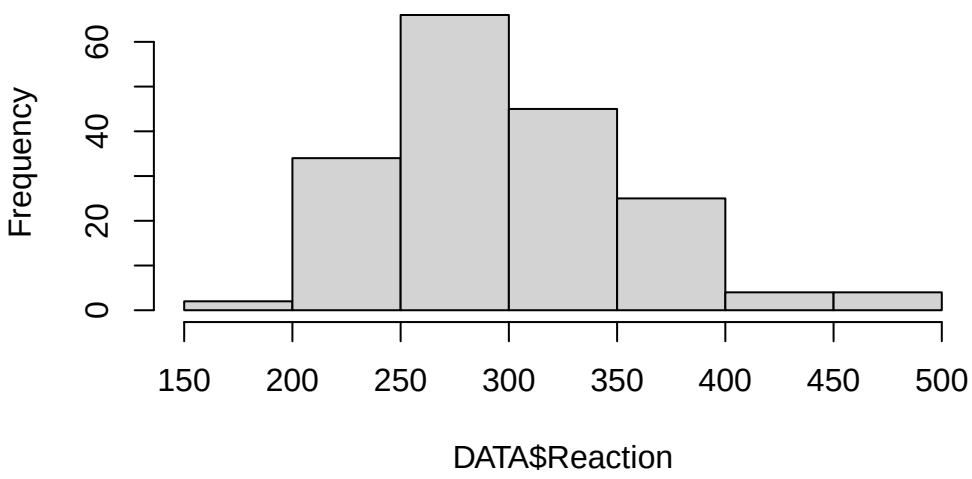
7.8 Exploratory Data Analysis

Base R:

```r
## observing the distributions of the data
plot( density( DATA$Reaction) )
```




```r
hist( DATA$Reaction )
```




```
quantile( DATA$Reaction )
```

```
      0%      25%      50%      75%     100%
194.3322 255.3758 288.6508 336.7521 466.3535
```

```
quantile( DATA$Reaction[DATA$Group == "A"] )
```

```
      0%      25%      50%      75%     100%
194.3322 242.9500 283.8494 318.0617 466.3535
```

```
quantile( DATA$Reaction[DATA$Group == "B"] )
```

```
      0%      25%      50%      75%     100%
221.6771 269.8009 304.6321 350.2706 458.9167
```

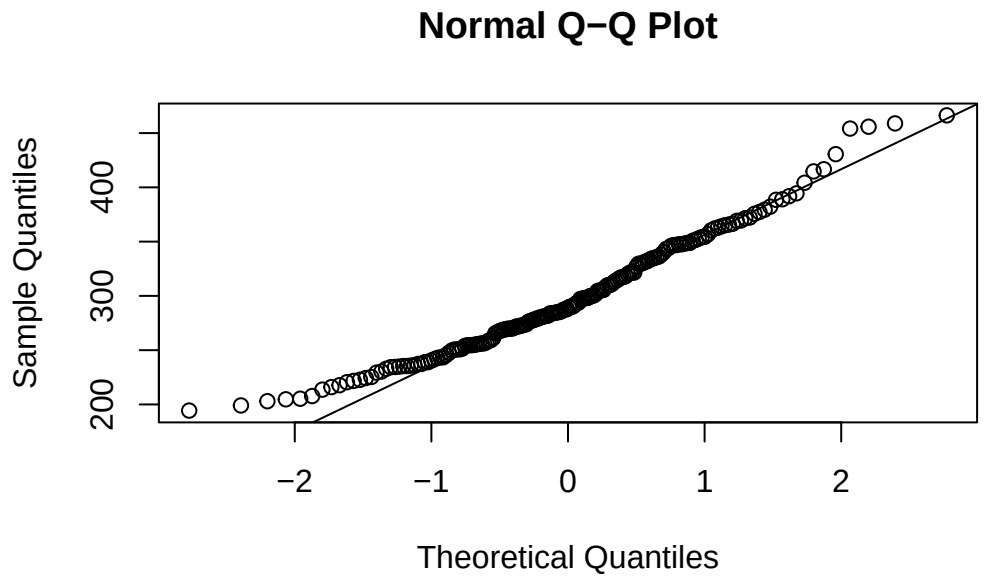
```
moments::skewness( DATA$Reaction )
```

```
[1] 0.5862549
```

```
# check the formula
# Generally 3 means the kurtosis is OK
#           in other cases 0 is OK (i.e. in library e1071)
moments::kurtosis( DATA$Reaction )
```

```
[1] 3.063391
```

```
## testing the normality of the distribution
qqnorm( DATA$Reaction )
qqline( DATA$Reaction )
```




```
shapiro.test( DATA$Reaction )
```

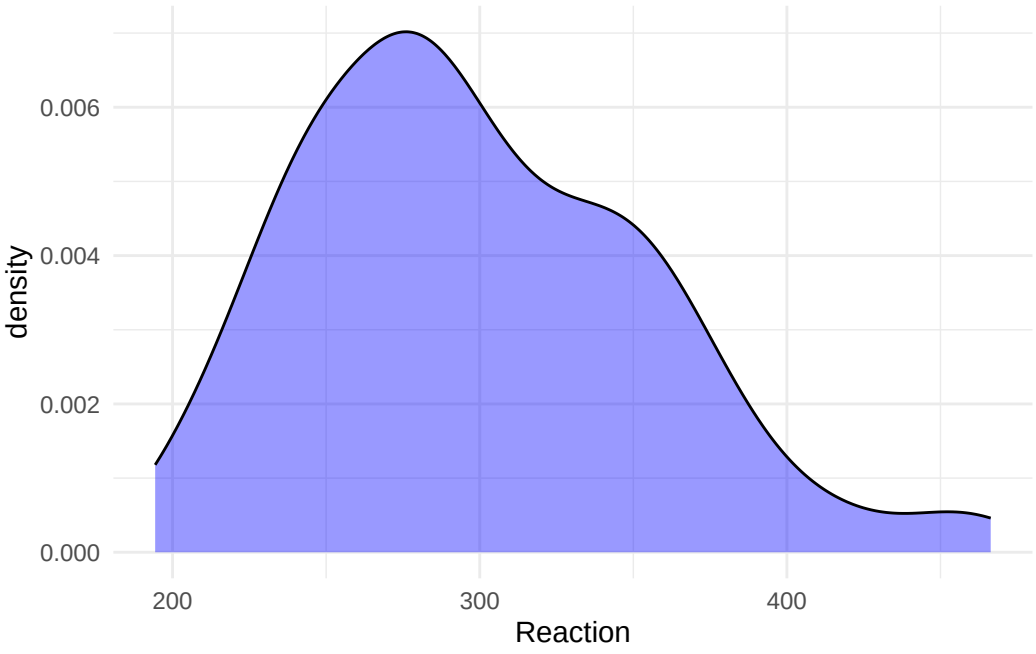
```
    Shapiro-Wilk normality test

data:  DATA$Reaction
W = 0.97074, p-value = 0.00078
```

. . .

tidyverse:

```
## observing the distributions of the data
ggplot(DATA, aes(x = Reaction)) +
  geom_density(fill = "blue", alpha = 0.4) +
  theme_minimal()
```

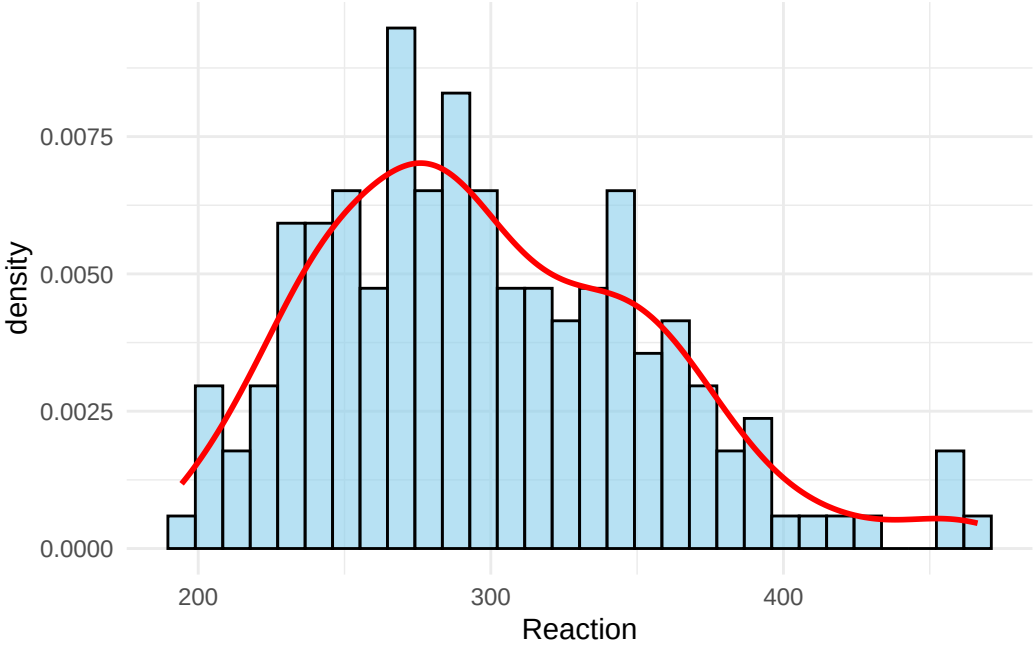



```
ggplot(DATA, aes(x = Reaction)) +
  geom_histogram(aes(y = ..density..), bins = 30,
```

```
                 fill = "skyblue", color = "black", alpha = 0.6) +
  geom_density(color = "red", size = 1) +
  theme_minimal()
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```

```
Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
i Please use `after_stat(density)` instead.
```




```
DATA %>%
 reframe(enframe(quantile(Reaction), "quantile", "Reaction"))
```

```
# A tibble: 5 x 2
  quantile Reaction
  <chr>       <dbl>
1 0%           194.
2 25%          255.
3 50%          289.
4 75%          337.
5 100%         466.
```

```
DATA %>%
 group_by(Group) %>%
 reframe(enframe(quantile(Reaction), "quantile", "Reaction"))
```

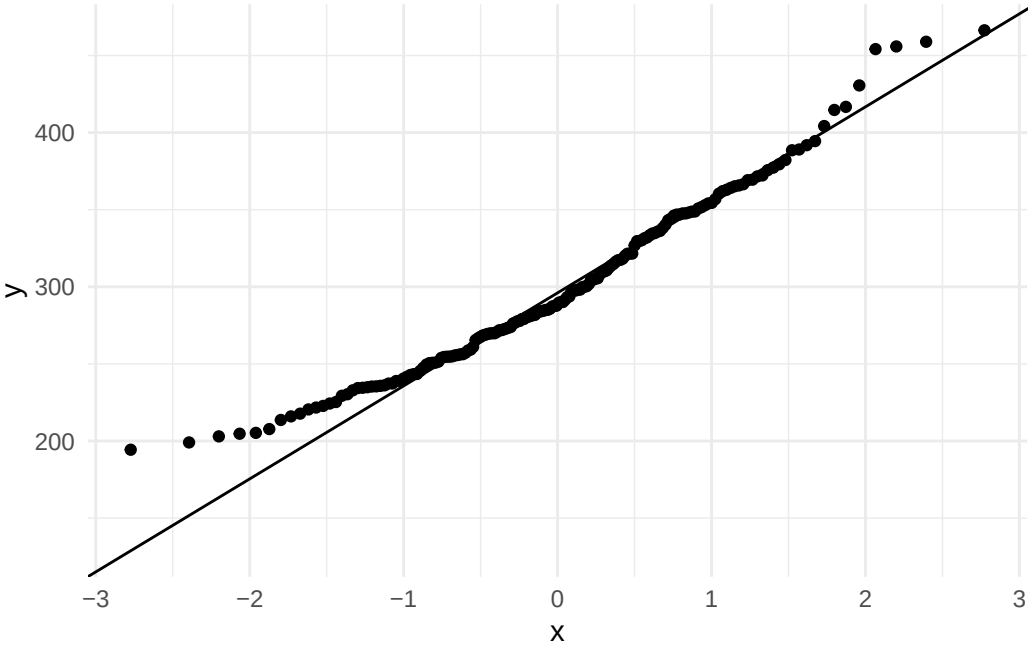
```
# A tibble: 10 x 3
   Group quantile Reaction
   <chr> <chr>       <dbl>
 1 A     0%            194.
 2 A     25%           243.
 3 A     50%           284.
 4 A     75%           318.
 5 A     100%          466.
 6 B     0%            222.
```

```
 7 B     25%          270.
 8 B     50%          305.
 9 B     75%          350.
10 B     100%         459.
```

```
DATA %>%
  reframe(
    skewness = moments::skewness(Reaction, na.rm = TRUE),
    kurtosis = moments::kurtosis(Reaction, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 2
  skewness kurtosis
     <dbl>    <dbl>
1    0.586     3.06
```

```
## testing the normality of the distribution
ggplot(DATA, aes(sample = Reaction)) +
  stat_qq() +
  stat_qq_line() +
  theme_minimal()
```



7.9 Checking for outliers

An outlier is a data point that is significantly different from the rest of the data.

It lies far outside the expected range, either much higher or lower than most other values.

Outliers can result from errors, variability, or rare events and may affect analysis results.

Base R:

```
## create criterion
mean_sd <- c(
  mean( DATA$Reaction ) - 2.5 * sd( DATA$Reaction ),
  mean( DATA$Reaction ) + 2.5 * sd( DATA$Reaction )
)
```

```r
perc <- quantile( DATA$Reaction, probs = c(0.001, 0.991) )

## find the rows of the database that appear to be outliers
out1 <- which(
    DATA$Reaction > mean_sd[2] | DATA$Reaction < mean_sd[1]
)
out1
```

```
[1]  10  57  99 100
```

```r
out2 <- which(
    DATA$Reaction > perc[2] | DATA$Reaction < perc[1]
)
out2
```

```
[1]  10  22 100
```

```r
## remove the outliers from the dataset
DATA1 <- droplevels( DATA[-out1,] )
DATA2 <- droplevels( DATA[-out2,] )

nrow(DATA)
```

```
[1] 180
```

```r
nrow(DATA1)
```

```
[1] 176
```

```r
nrow(DATA2)
```

```
[1] 177
```

. . .

tidyverse:

```r
# Define outlier criteria
DATA <- DATA %>%
  mutate(
    mean_sd_lower = mean(Reaction) - 2.5 * sd(Reaction),
    mean_sd_upper = mean(Reaction) + 2.5 * sd(Reaction),
    perc_lower = quantile(Reaction, probs = 0.001),
    perc_upper = quantile(Reaction, probs = 0.991)
  )

# Remove outliers based on mean ± 2.5 SD
DATA1 <- DATA %>%
  filter(Reaction >= mean_sd_lower & Reaction <= mean_sd_upper) %>%
  droplevels()

# Remove outliers based on quantile thresholds (0.1% and 99.1%)
DATA2 <- DATA %>%
  filter(Reaction >= perc_lower & Reaction <= perc_upper) %>%
  droplevels()
```

```r
# Display number of rows
nrow(DATA)
```

```
[1] 180
```

```r
nrow(DATA1)
```

```
[1] 176
```

```r
nrow(DATA2)
```

```
[1] 177
```

7.10 Statistical Analysis

```r
# Correlation matrix
cor(DATA[,c("Reaction", "Days")])
```

```
          Reaction      Days
Reaction 1.0000000 0.5352302
Days     0.5352302 1.0000000
```

```r
# Correlation test
cor.test(DATA$Reaction, DATA$Days)
```

```
    Pearson's product-moment correlation

data:  DATA$Reaction and DATA$Days
t = 8.4537, df = 178, p-value = 9.894e-15
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.4220032 0.6320164
sample estimates:
      cor
0.5352302
```

```r
# Linear regression
model <- lm(Reaction ~ Days * Group, data = DATA)

summary(model)
```

```
Call:
lm(formula = Reaction ~ Days * Group, data = DATA)

Residuals:
    Min      1Q  Median      3Q     Max
-87.104 -32.780  -0.098  25.501 157.539

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  252.292      8.872  28.437  < 2e-16 ***
Days           7.388      1.662   4.446 1.55e-05 ***
GroupB        -1.773     12.547  -0.141  0.88779
```

```
Days:GroupB    6.158      2.350   2.620  0.00956 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 45.28 on 176 degrees of freedom
Multiple R-squared:  0.3646,    Adjusted R-squared:  0.3537
F-statistic: 33.66 on 3 and 176 DF,  p-value: < 2.2e-16
```

```r
anova(model)
```

```
Analysis of Variance Table

Response: Reaction
            Df Sum Sq Mean Sq F value    Pr(>F)
Days         1 162703  162703 79.3442 6.385e-16 ***
Group        1  30271   30271 14.7620 0.0001701 ***
Days:Group   1  14076   14076  6.8645 0.0095607 **
Residuals  176 360904    2051
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# T-test (for comparing means)
t.test(DATA$Reaction[DATA$Group == "A"], DATA$Reaction[DATA$Group == "B"])
```

```
    Welch Two Sample t-test

data:  DATA$Reaction[DATA$Group == "A"] and DATA$Reaction[DATA$Group == "B"]
t = -3.1656, df = 177.39, p-value = 0.001822
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -42.104666  -9.767781
sample estimates:
mean of x mean of y
 285.5398  311.4760
```

```r
t.test(Reaction ~ Group, data = DATA)
```

```
    Welch Two Sample t-test

data:  Reaction by Group
t = -3.1656, df = 177.39, p-value = 0.001822
alternative hypothesis: true difference in means between group A and group B is not equal to 0
95 percent confidence interval:
 -42.104666  -9.767781
sample estimates:
mean in group A mean in group B
       285.5398        311.4760
```

```r
# Wilcoxon test (non-parametric data)
wilcox.test(DATA$Reaction[DATA$Group == "A"], DATA$Reaction[DATA$Group == "B"])
```

```
    Wilcoxon rank sum test with continuity correction
```

```
data:  DATA$Reaction[DATA$Group == "A"] and DATA$Reaction[DATA$Group == "B"]
W = 2915, p-value = 0.001171
alternative hypothesis: true location shift is not equal to 0
```

```r
wilcox.test(Reaction ~ Group, data = DATA)
```

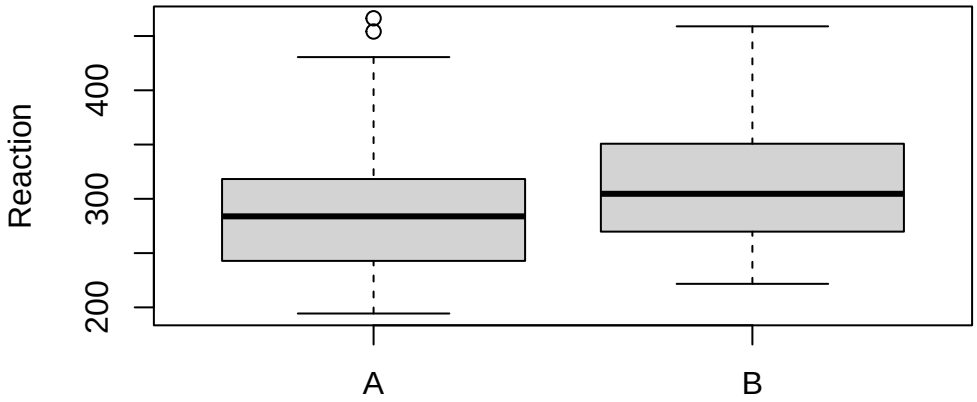
```
    Wilcoxon rank sum test with continuity correction

data:  Reaction by Group
W = 2915, p-value = 0.001171
alternative hypothesis: true location shift is not equal to 0
```

7.11 Data Visualization

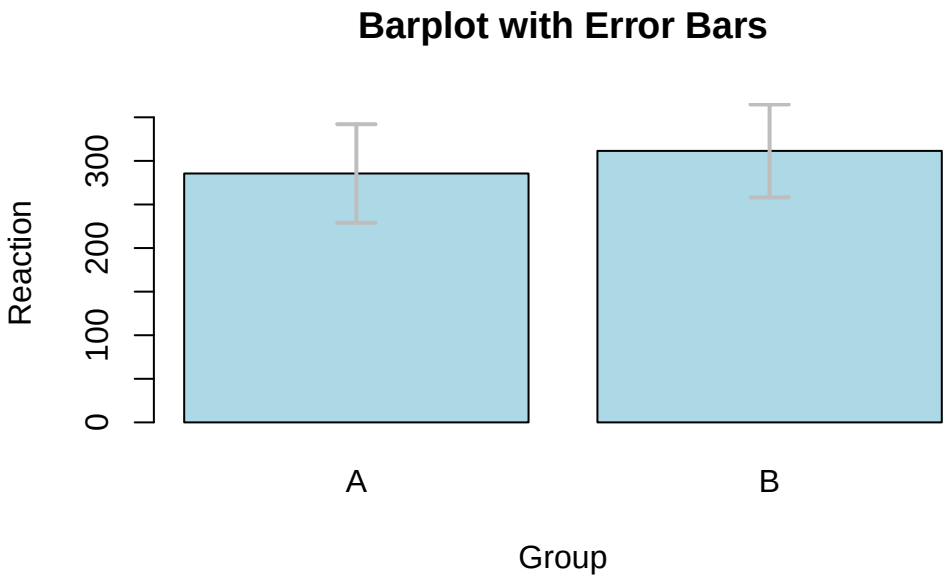
Base R:

```r
# Plot a Boxplot
boxplot(Reaction ~ Group, data = DATA)
```



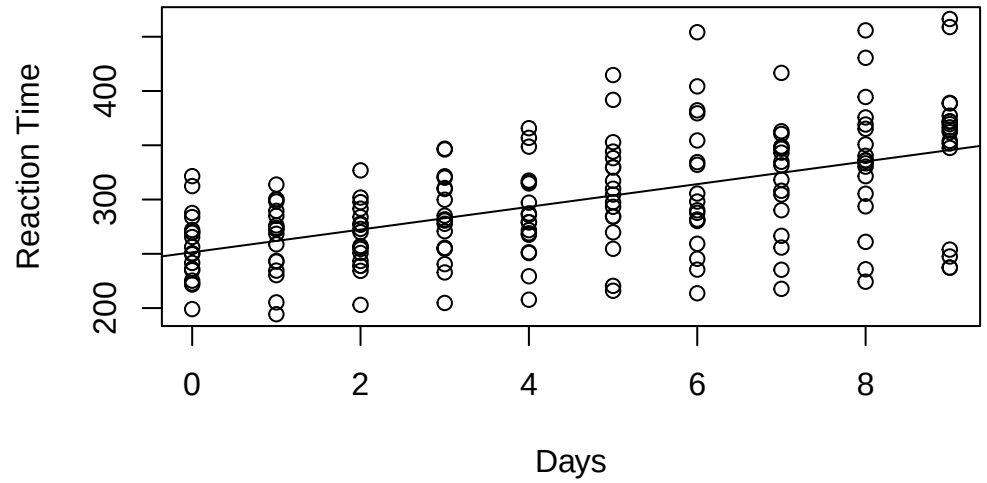

```r
# Plot a barplot
graph <- aggregate(Reaction ~ Group, data = DATA,
                   FUN = function(x){cbind(mean(x), sd(x))})

bar_positions <- barplot(Reaction[,1] ~ Group,
  data = graph,
                         ylim = c(0, max(graph$Reaction[,1] + graph$Reaction[,2]) + 1),
                         col = "lightblue",
                         main = "Barplot with Error Bars",
                         ylab = "Reaction")
# Add error bars
arrows(bar_positions, graph$Reaction[,1] - graph$Reaction[,2],
       bar_positions, graph$Reaction[,1] + graph$Reaction[,2],
       angle = 90, code = 3, length = 0.1, col = "grey", lwd = 2)
```




```
# Plot a Linear regression
model <- lm(Reaction ~ Days, data = DATA)

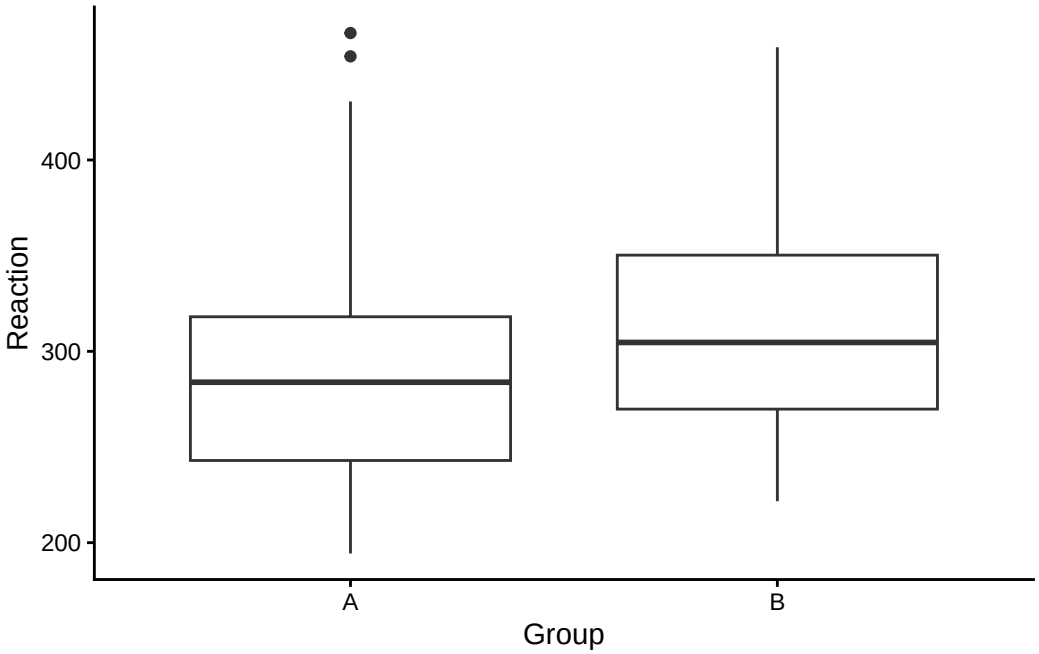
plot( y = DATA$Reaction, x = DATA$Days ,
      xlab = "Days", ylab = "Reaction Time")
abline( model )
```



. . .

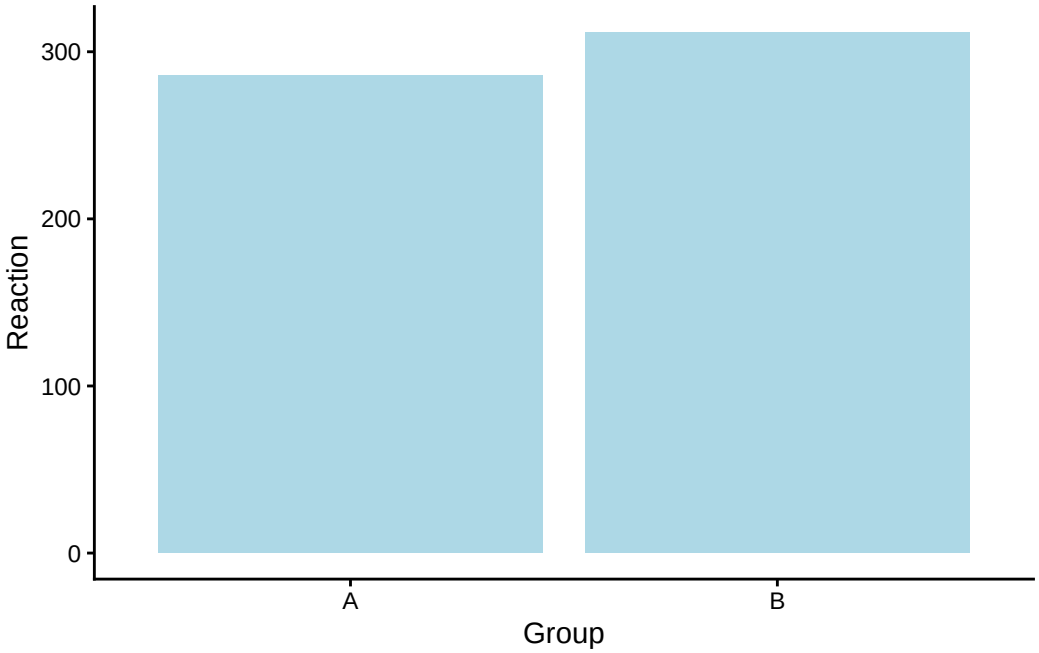
ggplot2:

```
# Plot a boxplot
ggplot(DATA, aes(y = Reaction, x = Group)) +
  geom_boxplot()+
  theme_classic()
```



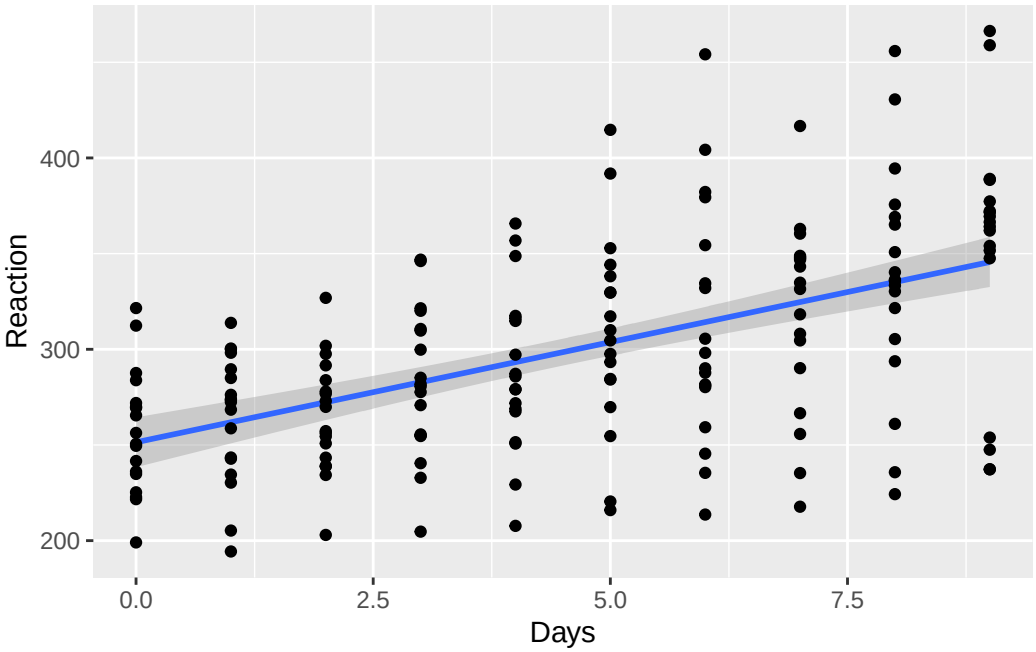

```
# Plot a barplot
ggplot(DATA, aes(y = Reaction, x = Group)) +
  stat_summary(geom = "bar", fun = mean, fill = "lightblue")+
  stat_summary(geom = "pointrange", fun.data = mean_sdl, colour = "grey")+
  theme_classic()
```

```
Warning: Computation failed in `stat_summary()`.
Caused by error in `fun.data()`:
! The package "Hmisc" is required.
```




```
# Plot a Linear regression
ggplot(DATA, aes(y = Reaction, x = Days)) +
  geom_smooth(method = "lm")+
  geom_point()
```

```
`geom_smooth()` using formula = 'y ~ x'
```



7.12 Combine all elements into a unique script - Base R

```
# This study examines the average reaction time per day (in milliseconds)
# for subjects in a sleep deprivation experiment.
# Participants were divided into two groups and asked to perform
# a speeded response task over multiple days in a longitudinal study.
# During the experiment, some participants were deprived of sleep for a
# certain number of days, allowing researchers to analyze the
# effects of sleep deprivation on reaction times.
##
## In the dataset the dependent variable is: Reaction,
##    that is the raction time in milliseconds
## In the dataset the independet variables are:
## a. Days (the number of days without sleeping)
## b. Group (A or B)

# rm(list = ls())
# graphics.off()

# setwd("path")

DATA <- read.csv(file = "data/data_sleepstudy.csv")

# Check the structure of the dataset
str(DATA)
```

```
'data.frame':   180 obs. of  5 variables:
 $ X       : int  1 2 3 4 5 6 7 8 9 10 ...
 $ Reaction: num  250 259 251 321 357 ...
 $ Days    : int  0 1 2 3 4 5 6 7 8 9 ...
 $ Subject : int  308 308 308 308 308 308 308 308 308 308 ...
```

```
 $ Group   : chr  "A" "A" "A" "A" ...
```

```
# View the first few rows
head(DATA)
```

```
  X Reaction Days Subject Group
1 1 249.5600    0     308     A
2 2 258.7047    1     308     A
3 3 250.8006    2     308     A
4 4 321.4398    3     308     A
5 5 356.8519    4     308     A
6 6 414.6901    5     308     A
```

```
# Check for missing values and quantiles
summary(DATA)
```

```
       X             Reaction          Days        Subject
 Min.   :  1.00   Min.   :194.3   Min.   :0.0   Min.   :308.0
 1st Qu.: 45.75   1st Qu.:255.4   1st Qu.:2.0   1st Qu.:331.0
 Median : 90.50   Median :288.7   Median :4.5   Median :336.0
 Mean   : 90.50   Mean   :298.5   Mean   :4.5   Mean   :341.3
 3rd Qu.:135.25   3rd Qu.:336.8   3rd Qu.:7.0   3rd Qu.:352.0
 Max.   :180.00   Max.   :466.4   Max.   :9.0   Max.   :372.0
    Group
 Length:180
 Class :character
 Mode  :character
```

```
quantile(DATA$Reaction)
```

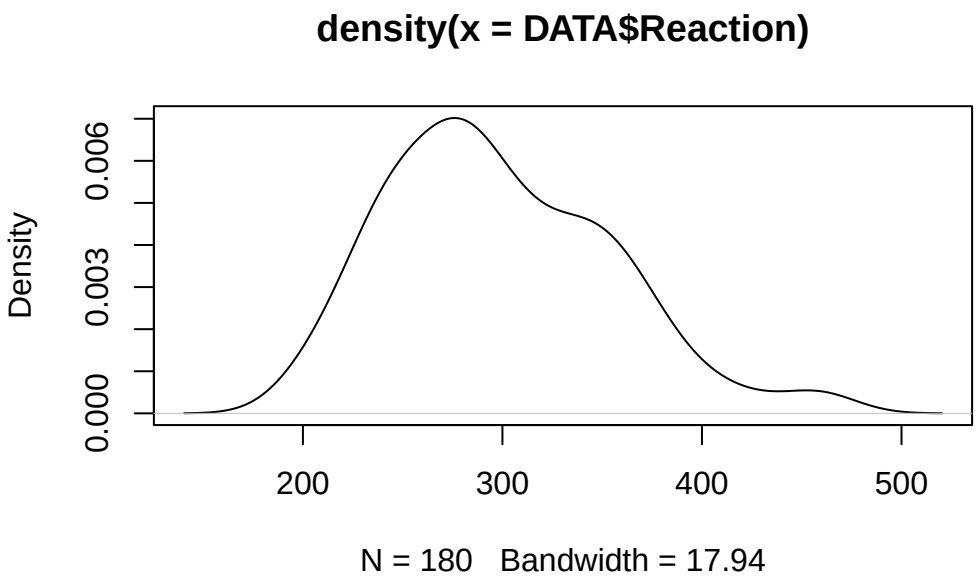
```
      0%      25%      50%      75%     100%
194.3322 255.3758 288.6508 336.7521 466.3535
```

```
quantile(DATA$Days)
```

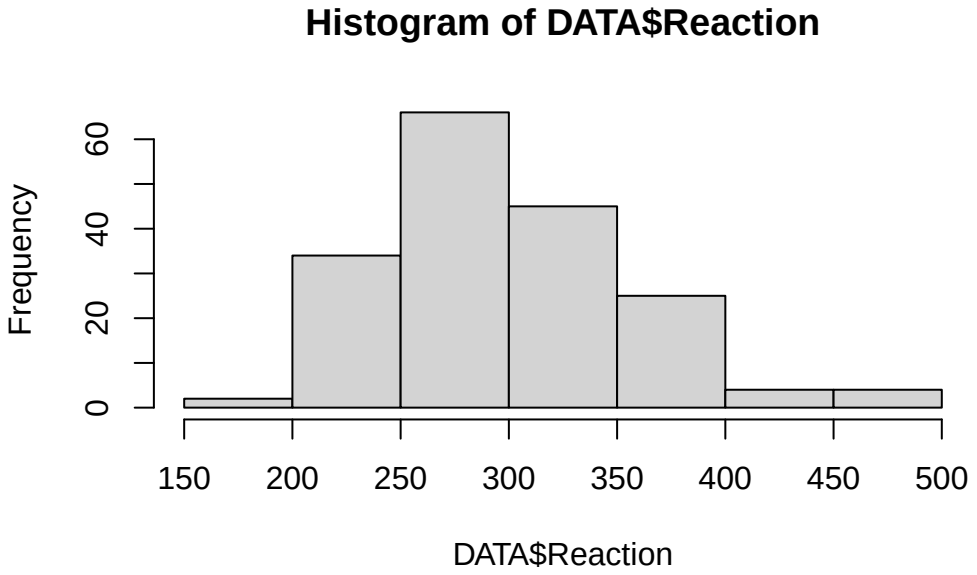
```
  0%  25%  50%  75% 100%
 0.0  2.0  4.5  7.0  9.0
```

```
# Handling missing values
DATA <- na.omit(DATA)  # Removes rows with NA values

## observing the distributions of the data
plot( density( DATA$Reaction) )
```




```
hist( DATA$Reaction )
```




```
quantile( DATA$Reaction )
```

```
      0%      25%      50%      75%     100%
194.3322 255.3758 288.6508 336.7521 466.3535
```

```
quantile( DATA$Reaction[DATA$Group == "A"] )
```

```
      0%      25%      50%      75%     100%
194.3322 242.9500 283.8494 318.0617 466.3535
```

```
quantile( DATA$Reaction[DATA$Group == "B"] )
```

```
      0%      25%      50%      75%     100%
221.6771 269.8009 304.6321 350.2706 458.9167
```

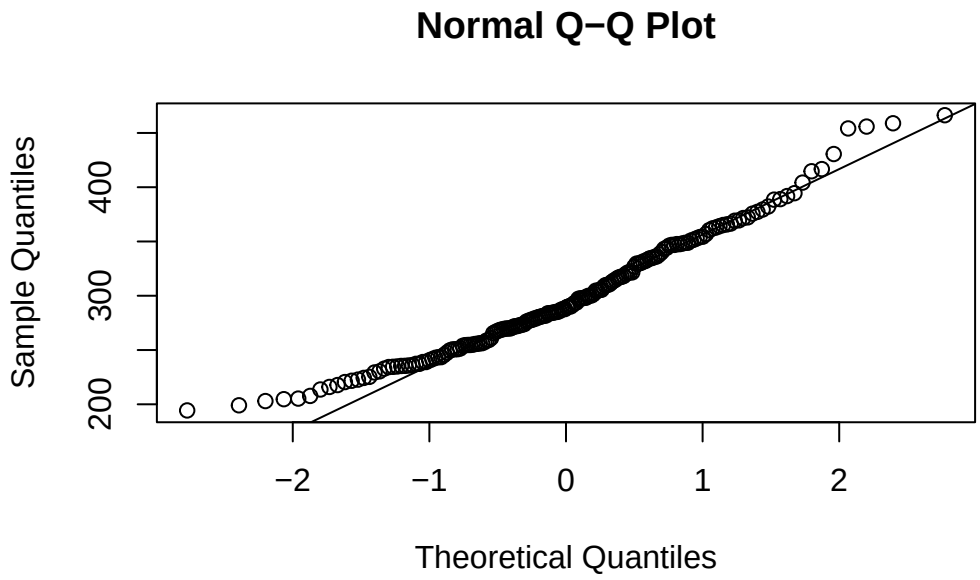
```
moments::skewness( DATA$Reaction )
```

```
[1] 0.5862549
```

```
# check the formula
# Generally 3 means the kurtosis is OK
#           in other cases 0 is OK (i.e. in library e1071)
moments::kurtosis( DATA$Reaction )
```

```
[1] 3.063391
```

```
## testing the normality of the distribution
qqnorm( DATA$Reaction )
qqline( DATA$Reaction )
```




```
shapiro.test( DATA$Reaction )
```

```
    Shapiro-Wilk normality test

data:  DATA$Reaction
W = 0.97074, p-value = 0.00078
```

```
## Checking for outliers

## create criterion
mean_sd <- c(
  mean( DATA$Reaction ) - 2.5 * sd( DATA$Reaction ),
  mean( DATA$Reaction ) + 2.5 * sd( DATA$Reaction )
)

## find the rows of the database that appear to be outliers
out1 <- which(
    DATA$Reaction > mean_sd[2] | DATA$Reaction < mean_sd[1]
)
out1
```

```
[1]  10  57  99 100
```

```
## % of outliers
length(out1)/nrow(DATA) * 100
```

```
[1] 2.222222
```

```
## Statistical Analysis

# Correlation test
cor.test(DATA$Reaction, DATA$Days)
```

```

    Pearson's product-moment correlation

data:  DATA$Reaction and DATA$Days
t = 8.4537, df = 178, p-value = 9.894e-15
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.4220032 0.6320164
sample estimates:
      cor
0.5352302
```

```
cor.test(DATA$Reaction[DATA$Group == "A"], DATA$Days[DATA$Group == "A"])
```

```

    Pearson's product-moment correlation

data:  DATA$Reaction[DATA$Group == "A"] and DATA$Days[DATA$Group == "A"]
t = 3.8227, df = 88, p-value = 0.0002459
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.1847127 0.5420951
sample estimates:
      cor
0.3773686
```

```
cor.test(DATA$Reaction[DATA$Group == "B"], DATA$Days[DATA$Group == "B"])
```

```

    Pearson's product-moment correlation

data:  DATA$Reaction[DATA$Group == "B"] and DATA$Days[DATA$Group == "B"]
t = 10.131, df = 88, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6210377 0.8167421
sample estimates:
     cor
0.733759
```

```
# Linear regression
model <- lm(Reaction ~ Days * Group, data = DATA)

summary(model)
```

```

Call:
lm(formula = Reaction ~ Days * Group, data = DATA)
```

```
Residuals:
    Min      1Q  Median      3Q     Max
-87.104 -32.780  -0.098  25.501 157.539

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  252.292      8.872  28.437  < 2e-16 ***
Days           7.388      1.662   4.446 1.55e-05 ***
GroupB        -1.773     12.547  -0.141  0.88779
Days:GroupB    6.158      2.350   2.620  0.00956 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 45.28 on 176 degrees of freedom
Multiple R-squared:  0.3646,    Adjusted R-squared:  0.3537
F-statistic: 33.66 on 3 and 176 DF,  p-value: < 2.2e-16
```

```
anova(model)
```

```
Analysis of Variance Table

Response: Reaction
            Df Sum Sq Mean Sq F value    Pr(>F)
Days         1 162703  162703 79.3442 6.385e-16 ***
Group        1  30271   30271 14.7620 0.0001701 ***
Days:Group   1  14076   14076  6.8645 0.0095607 **
Residuals  176 360904    2051
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# T-test (for comparing means)
t.test(DATA$Reaction[DATA$Group == "A"], DATA$Reaction[DATA$Group == "B"])
```

```
    Welch Two Sample t-test

data:  DATA$Reaction[DATA$Group == "A"] and DATA$Reaction[DATA$Group == "B"]
t = -3.1656, df = 177.39, p-value = 0.001822
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -42.104666  -9.767781
sample estimates:
mean of x mean of y
 285.5398  311.4760
```

```
t.test(Reaction ~ Group, data = DATA)
```

```
    Welch Two Sample t-test

data:  Reaction by Group
t = -3.1656, df = 177.39, p-value = 0.001822
alternative hypothesis: true difference in means between group A and group B is not equal to
```

```
95 percent confidence interval:
 -42.104666  -9.767781
sample estimates:
mean in group A mean in group B
       285.5398        311.4760
```

```r
# Wilcoxon test (non-parametric data)
wilcox.test(DATA$Reaction[DATA$Group == "A"], DATA$Reaction[DATA$Group == "B"])
```

```
    Wilcoxon rank sum test with continuity correction

data:  DATA$Reaction[DATA$Group == "A"] and DATA$Reaction[DATA$Group == "B"]
W = 2915, p-value = 0.001171
alternative hypothesis: true location shift is not equal to 0
```

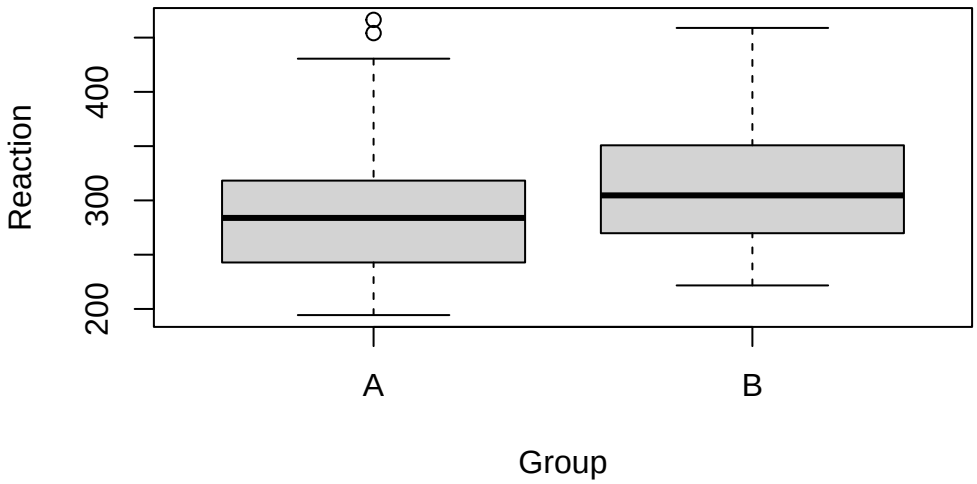
```r
wilcox.test(Reaction ~ Group, data = DATA)
```

```
    Wilcoxon rank sum test with continuity correction

data:  Reaction by Group
W = 2915, p-value = 0.001171
alternative hypothesis: true location shift is not equal to 0
```

```r
## Data Visualization

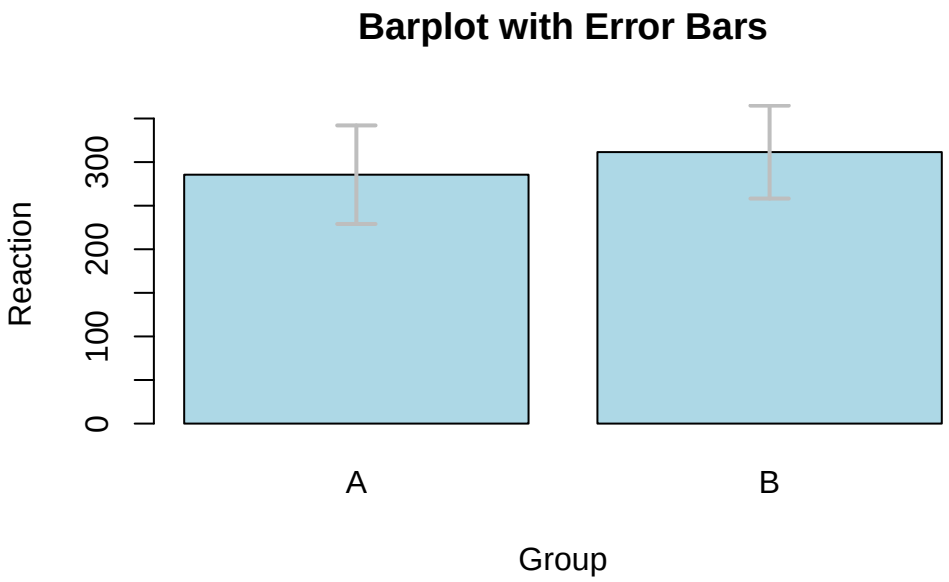
# Plot a Boxplot
boxplot(Reaction ~ Group, data = DATA)
```




```r
# Plot a barplot
graph <- aggregate(Reaction ~ Group, data = DATA,
                   FUN = function(x){cbind(mean(x), sd(x))})

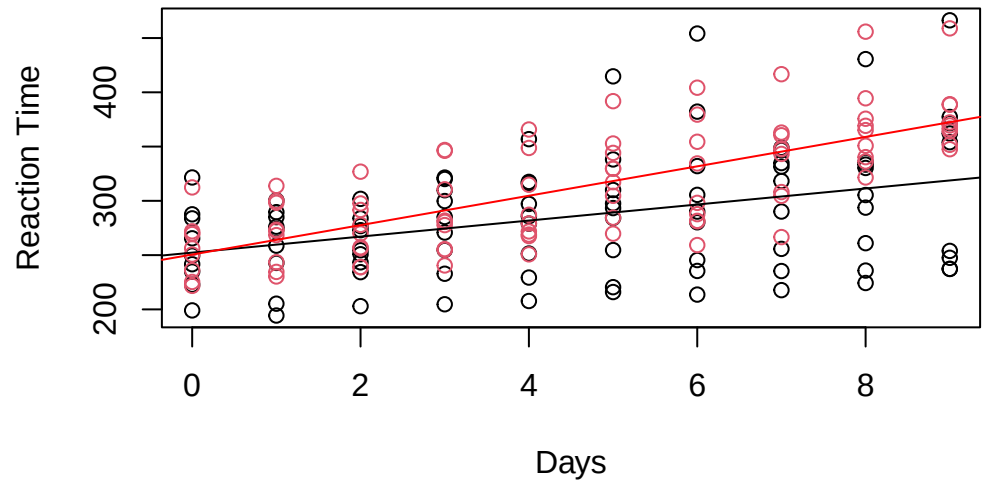
bar_positions <- barplot(Reaction[,1] ~ Group,
  data = graph,
                         ylim = c(0, max(graph$Reaction[,1] + graph$Reaction[,2]) + 1),
                         col = "lightblue",
                         main = "Barplot with Error Bars",
                         ylab = "Reaction")
```

```
# Add error bars
arrows(bar_positions, graph$Reaction[,1] - graph$Reaction[,2],
       bar_positions, graph$Reaction[,1] + graph$Reaction[,2],
       angle = 90, code = 3, length = 0.1, col = "grey", lwd = 2)
```




```
# Plot a Linear regression
modelA <- lm(Reaction ~ Days, data = DATA[DATA$Group == "A",])
modelB <- lm(Reaction ~ Days, data = DATA[DATA$Group == "B",])

plot( y = DATA$Reaction, x = DATA$Days ,
      col = as.factor(DATA$Group),
      xlab = "Days", ylab = "Reaction Time")
abline( modelA )
abline( modelB, col = "red" )
```



7.13 Combine all elements into a unique script - `tidyverse`

```
# This study examines the average reaction time per day (in milliseconds)
# for subjects in a sleep deprivation experiment.
# Participants were divided into two groups and asked to perform
# a speeded response task over multiple days in a longitudinal study.
```

```r
# During the experiment, some participants were deprived of sleep for a
# certain number of days, allowing researchers to analyze the
# effects of sleep deprivation on reaction times.
##
## In the dataset the dependent variable is: Reaction,
##    that is the raction time in milliseconds
## In the dataset the independet variables are:
## a. Days (the number of days without sleeping)
## b. Group (A or B)

# rm(list = ls())
# graphics.off()

library(tidyverse)
library(ggplot2)
library(dplyr)

# setwd("path")

DATA <- read.csv(file = "data/data_sleepstudy.csv")

# Check the structure of the dataset
str(DATA)
```

```
'data.frame':   180 obs. of  5 variables:
 $ X       : int  1 2 3 4 5 6 7 8 9 10 ...
 $ Reaction: num  250 259 251 321 357 ...
 $ Days    : int  0 1 2 3 4 5 6 7 8 9 ...
 $ Subject : int  308 308 308 308 308 308 308 308 308 308 ...
 $ Group   : chr  "A" "A" "A" "A" ...
```

```r
# View the first few rows
glimpse(DATA)
```

```
Rows: 180
Columns: 5
$ X        <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18~
$ Reaction <dbl> 249.5600, 258.7047, 250.8006, 321.4398, 356.8519, 414.6901, 3~
$ Days     <int> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0~
$ Subject  <int> 308, 308, 308, 308, 308, 308, 308, 308, 308, 308, 309, 309, 3~
$ Group    <chr> "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "~
```

```r
skimr::skim(DATA)
```

Tabella 3: Data summary

Name	DATA
Number of rows	180
Number of columns	5
Column type frequency:	
character	1
numeric	4

Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Group	0	1	1	1	0	2	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
X	0	1	90.50	52.11	1.00	45.75	90.50	135.25	180.00	
Reaction	0	1	298.51	56.33	194.33	255.38	288.65	336.75	466.35	
Days	0	1	4.50	2.88	0.00	2.00	4.50	7.00	9.00	
Subject	0	1	341.28	20.32	308.00	331.00	336.00	352.00	372.00	

```
DATA %>%
  summarise(across(everything(), ~ sum(is.na(.)), .names = "missing_{.col}"))
```

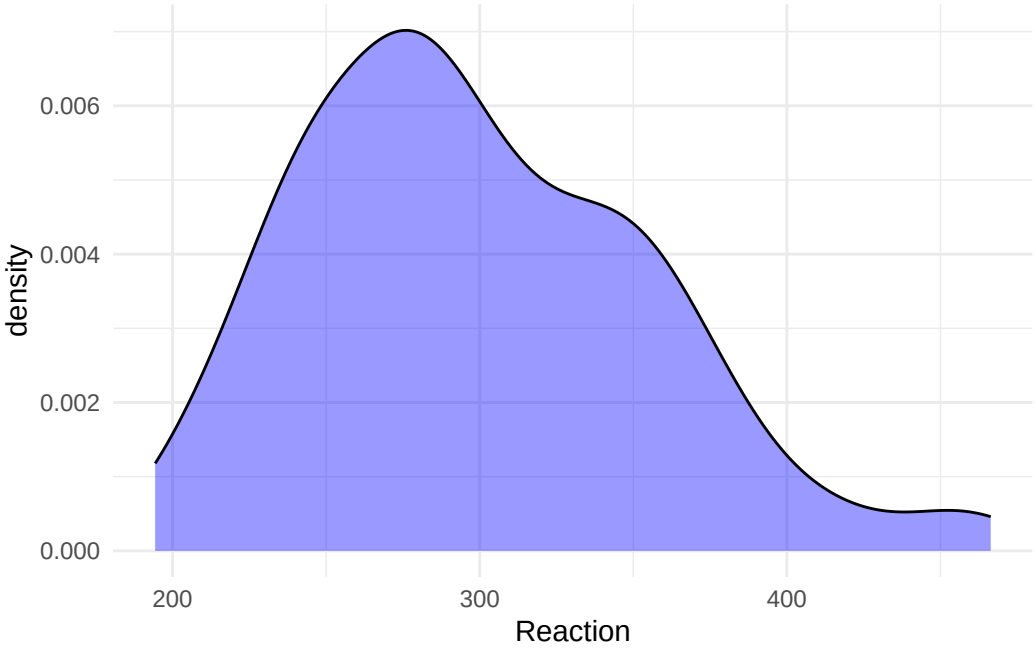
```
  missing_X missing_Reaction missing_Days missing_Subject missing_Group
1         0                0            0               0             0
```

```
DATA %>%
  summarise(across(where(is.numeric),
                   list(
                     mean = mean,
                     sd = sd,
                     min = min,
                     max = max),
                   na.rm = TRUE)
            )
```

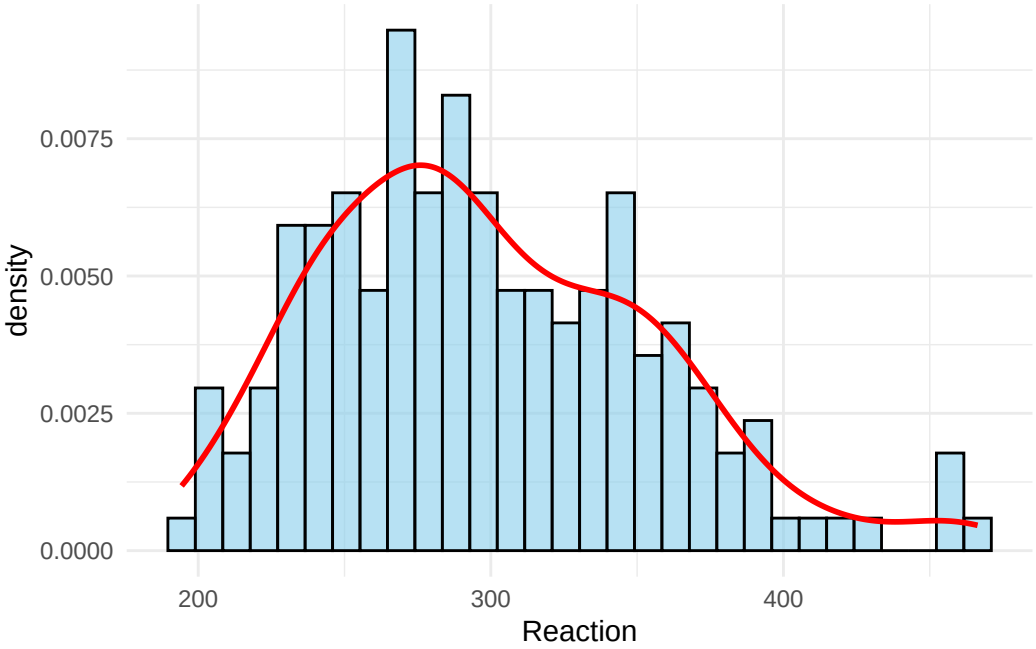
```
  X_mean     X_sd X_min X_max Reaction_mean Reaction_sd Reaction_min
1   90.5 52.10566     1   180      298.5079    56.32876     194.3322
  Reaction_max Days_mean  Days_sd Days_min Days_max Subject_mean Subject_sd
1     466.3535       4.5 2.880293        0        9     341.2778    20.3209
  Subject_min Subject_max
1         308         372
```

```
# Handling missing values
DATA <- DATA %>% drop_na()  # Removes rows with NA values

## observing the distributions of the data
ggplot(DATA, aes(x = Reaction)) +
  geom_density(fill = "blue", alpha = 0.4) +
  theme_minimal()
```




```
ggplot(DATA, aes(x = Reaction)) +
  geom_histogram(aes(y = ..density..), bins = 30,
                 fill = "skyblue", color = "black", alpha = 0.6) +
  geom_density(color = "red", size = 1) +
  theme_minimal()
```




```
DATA %>%
 reframe(enframe(quantile(Reaction), "quantile", "Reaction"))
```

```
  quantile Reaction
1       0% 194.3322
2      25% 255.3758
3      50% 288.6508
4      75% 336.7521
5     100% 466.3535
```

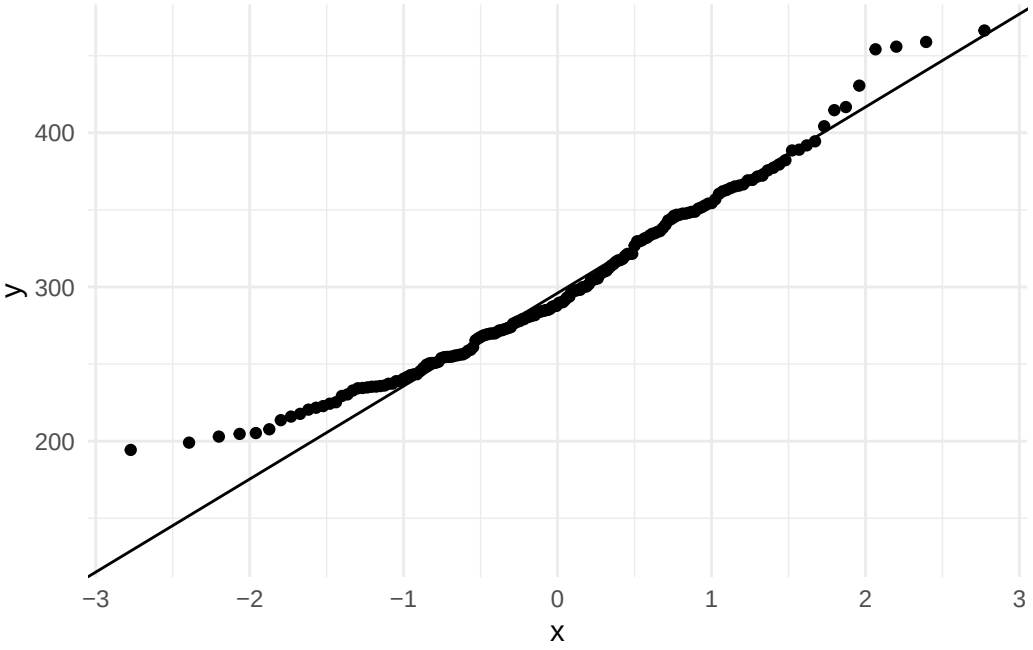
```r
DATA %>%
 group_by(Group) %>%
 reframe(enframe(quantile(Reaction), "quantile", "Reaction"))
```

```
# A tibble: 10 x 3
   Group quantile Reaction
   <chr> <chr>       <dbl>
 1 A     0%           194.
 2 A     25%          243.
 3 A     50%          284.
 4 A     75%          318.
 5 A     100%         466.
 6 B     0%           222.
 7 B     25%          270.
 8 B     50%          305.
 9 B     75%          350.
10 B     100%         459.
```

```r
DATA %>%
  reframe(
    skewness = moments::skewness(Reaction, na.rm = TRUE),
    kurtosis = moments::kurtosis(Reaction, na.rm = TRUE)
  )
```

```
   skewness kurtosis
1 0.5862549 3.063391
```

```r
## testing the normality of the distribution
ggplot(DATA, aes(sample = Reaction)) +
  stat_qq() +
  stat_qq_line() +
  theme_minimal()
```




```r
shapiro.test( DATA$Reaction )
```

```
    Shapiro-Wilk normality test

data:  DATA$Reaction
W = 0.97074, p-value = 0.00078
```

```
## Checking for outliers

# Define outlier criteria
DATA <- DATA %>%
  mutate(
    mean_sd_lower = mean(Reaction) - 2.5 * sd(Reaction),
    mean_sd_upper = mean(Reaction) + 2.5 * sd(Reaction),
    perc_lower = quantile(Reaction, probs = 0.001),
    perc_upper = quantile(Reaction, probs = 0.991)
  )

# Remove outliers based on mean ± 2.5 SD
DATA1 <- DATA %>%
  filter(Reaction >= mean_sd_lower & Reaction <= mean_sd_upper) %>%
  droplevels()

## % of outliers
(1-length(DATA1)/nrow(DATA)) * 100
```

```
[1] 95
```

```
DATA <- DATA1
rm(DATA1)

## Statistical Analysis

# Correlation test
cor.test(DATA$Reaction, DATA$Days)
```

```
    Pearson's product-moment correlation

data:  DATA$Reaction and DATA$Days
t = 7.9162, df = 174, p-value = 2.76e-13
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.3968640 0.6156364
sample estimates:
      cor
0.5145763
```

```
cor.test(DATA$Reaction[DATA$Group == "A"], DATA$Days[DATA$Group == "A"])
```

```
    Pearson's product-moment correlation

data:  DATA$Reaction[DATA$Group == "A"] and DATA$Days[DATA$Group == "A"]
t = 3.4388, df = 86, p-value = 0.0009031
```

```
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.1490968 0.5193085
sample estimates:
     cor
0.347682
```

```
cor.test(DATA$Reaction[DATA$Group == "B"], DATA$Days[DATA$Group == "B"])
```

```

    Pearson's product-moment correlation

data:  DATA$Reaction[DATA$Group == "B"] and DATA$Days[DATA$Group == "B"]
t = 9.82, df = 86, p-value = 1.056e-15
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6105760 0.8127287
sample estimates:
      cor
0.7270437
```

```
# Linear regression
model <- lm(Reaction ~ Days * Group, data = DATA)

summary(model)
```

```

Call:
lm(formula = Reaction ~ Days * Group, data = DATA)

Residuals:
    Min      1Q  Median      3Q     Max
-79.529 -31.263   1.827  26.948 129.651

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  254.498      8.129  31.307  < 2e-16 ***
Days           6.108      1.541   3.963 0.000108 ***
GroupB        -1.431     11.505  -0.124 0.901160
Days:GroupB    6.387      2.188   2.920 0.003975 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 41.35 on 172 degrees of freedom
Multiple R-squared:  0.3646,    Adjusted R-squared:  0.3535
F-statistic:  32.9 on 3 and 172 DF,  p-value: < 2.2e-16
```

```
anova(model)
```

```
Analysis of Variance Table

Response: Reaction
            Df Sum Sq Mean Sq F value    Pr(>F)
Days         1 122524  122524  71.675 1.079e-14 ***
```

```
Group        1  31606   31606  18.489 2.857e-05 ***
Days:Group   1  14571   14571   8.524  0.003975 **
Residuals  172 294024    1709
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# T-test (for comparing means)
t.test(DATA$Reaction[DATA$Group == "A"], DATA$Reaction[DATA$Group == "B"])
```

```
    Welch Two Sample t-test

data:  DATA$Reaction[DATA$Group == "A"] and DATA$Reaction[DATA$Group == "B"]
t = -3.5415, df = 173.85, p-value = 0.0005108
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -41.41018 -11.77151
sample estimates:
mean of x mean of y
 281.5689  308.1598
```

```r
t.test(Reaction ~ Group, data = DATA)
```

```
    Welch Two Sample t-test

data:  Reaction by Group
t = -3.5415, df = 173.85, p-value = 0.0005108
alternative hypothesis: true difference in means between group A and group B is not equal to 0
95 percent confidence interval:
 -41.41018 -11.77151
sample estimates:
mean in group A mean in group B
       281.5689        308.1598
```

```r
# Wilcoxon test (non-parametric data)
wilcox.test(DATA$Reaction[DATA$Group == "A"], DATA$Reaction[DATA$Group == "B"])
```

```
    Wilcoxon rank sum test with continuity correction

data:  DATA$Reaction[DATA$Group == "A"] and DATA$Reaction[DATA$Group == "B"]
W = 2737, p-value = 0.0007885
alternative hypothesis: true location shift is not equal to 0
```

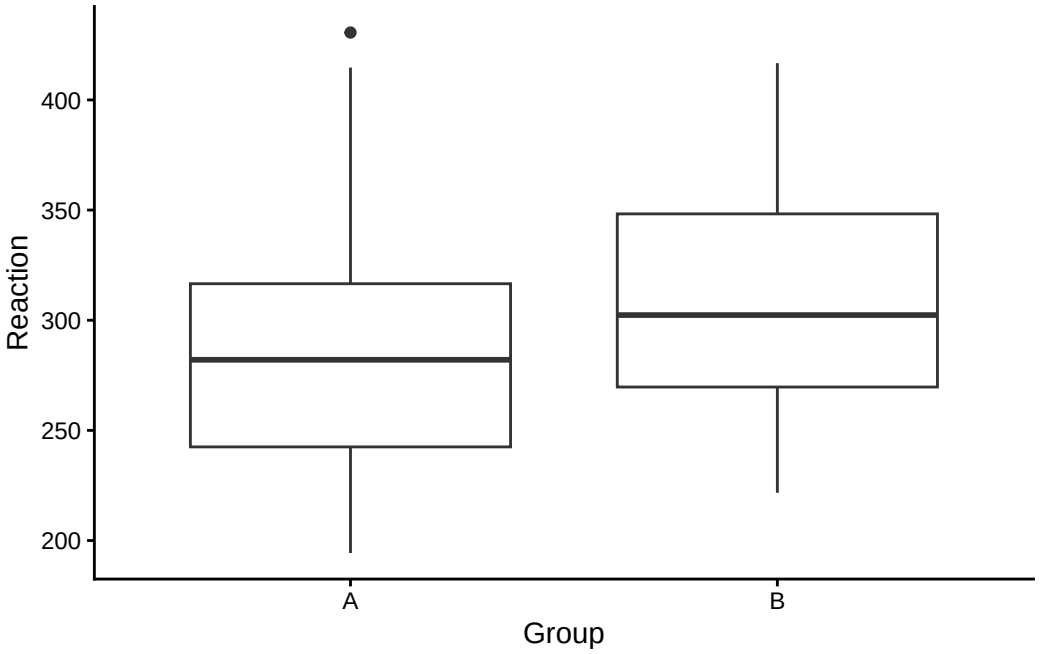
```r
wilcox.test(Reaction ~ Group, data = DATA)
```

```
    Wilcoxon rank sum test with continuity correction

data:  Reaction by Group
W = 2737, p-value = 0.0007885
alternative hypothesis: true location shift is not equal to 0
```

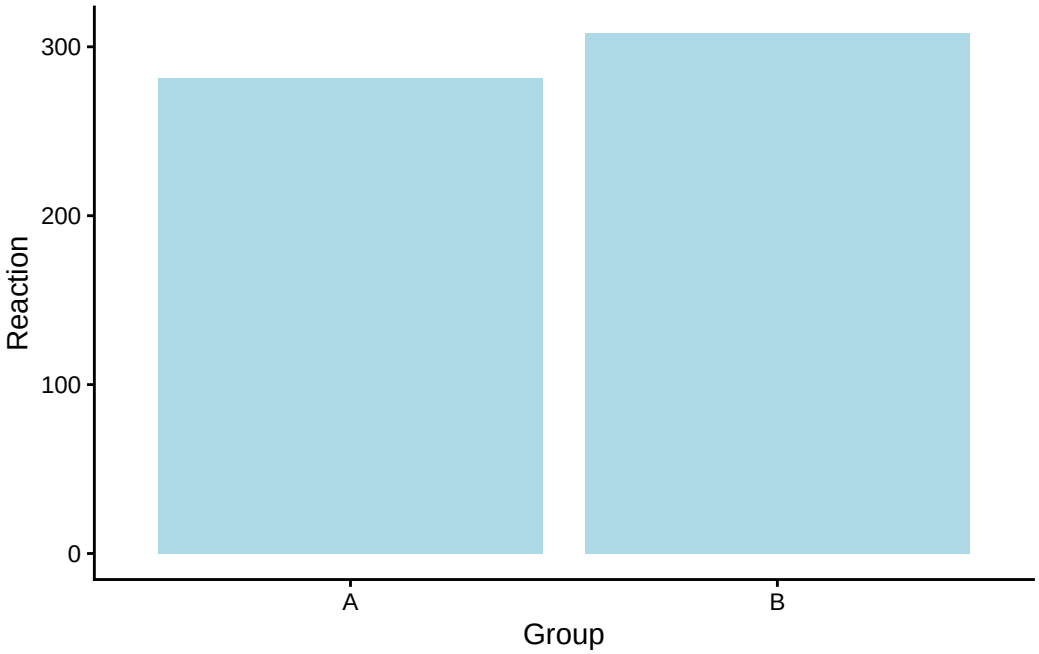
```
## Data Visualization

# Plot a boxplot
ggplot(DATA, aes(y = Reaction, x = Group)) +
  geom_boxplot()+
  theme_classic()
```



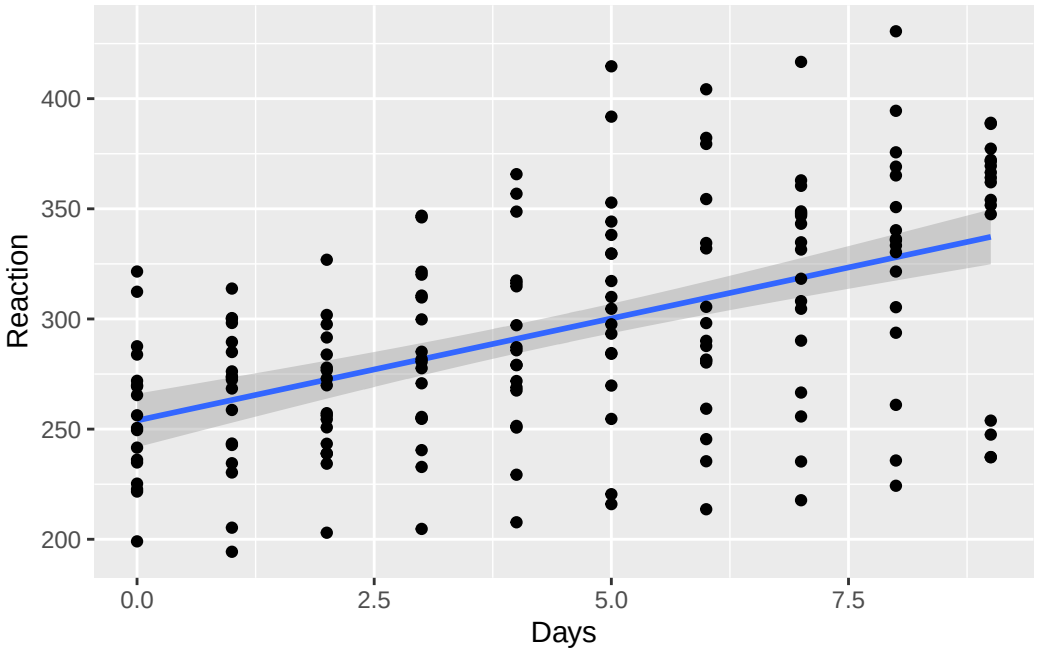

```
# Plot a barplot
ggplot(DATA, aes(y = Reaction, x = Group)) +
  stat_summary(geom = "bar", fun = mean, fill = "lightblue")+
  stat_summary(geom = "pointrange", fun.data = mean_sdl, colour = "grey")+
  theme_classic()
```

```
Warning: Computation failed in `stat_summary()`.
Caused by error in `fun.data()`:
! The package "Hmisc" is required.
```




```
# Plot a Linear regression
ggplot(DATA, aes(y = Reaction, x = Days)) +
  geom_smooth(method = "lm")+
  geom_point()
```

```
`geom_smooth()` using formula = 'y ~ x'
```



8 Focus on statistical tests

8.1 Correlation

8.1.1 What It Measures

- **Strength and direction** of a linear relationship between two continuous variables.
- Common metrics:

- `pearson`: parametric
- `spearman` or `kendall`: non-parametric

8.1.2 Real Dataset Example: `mtcars`

```r
cor.test(mtcars$wt, mtcars$mpg, method = "pearson")
```

```
    Pearson's product-moment correlation

data:  mtcars$wt and mtcars$mpg
t = -9.559, df = 30, p-value = 1.294e-10
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.9338264 -0.7440872
sample estimates:
       cor
-0.8676594
```

```r
correlation::correlation(data = mtcars, select = c("wt", "mpg"))
```

```
# Correlation Matrix (pearson-method)

Parameter1 | Parameter2 |     r |         95% CI | t(30) |         p
-------------------------------------------------------------------
wt         |        mpg | -0.87 | [-0.93, -0.74] | -9.56 | < .001***

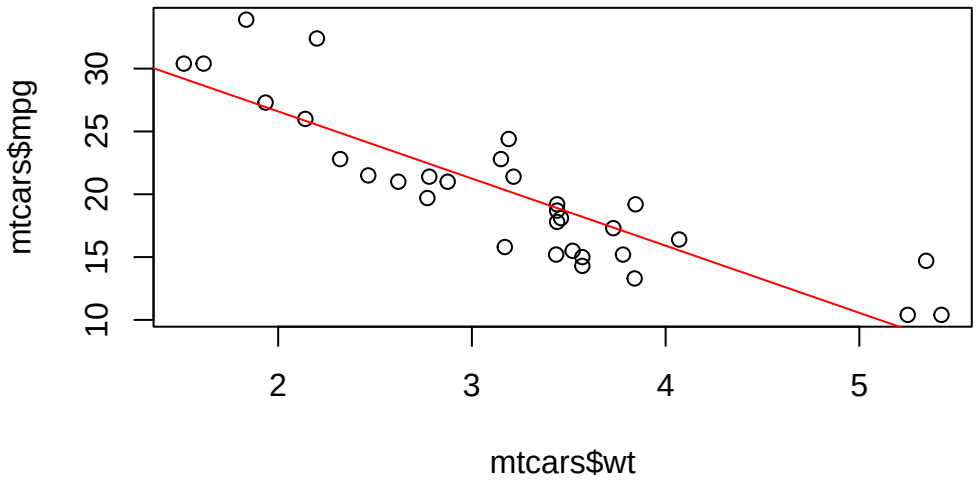
p-value adjustment method: Holm (1979)
Observations: 32
```

8.1.3 Assumptions

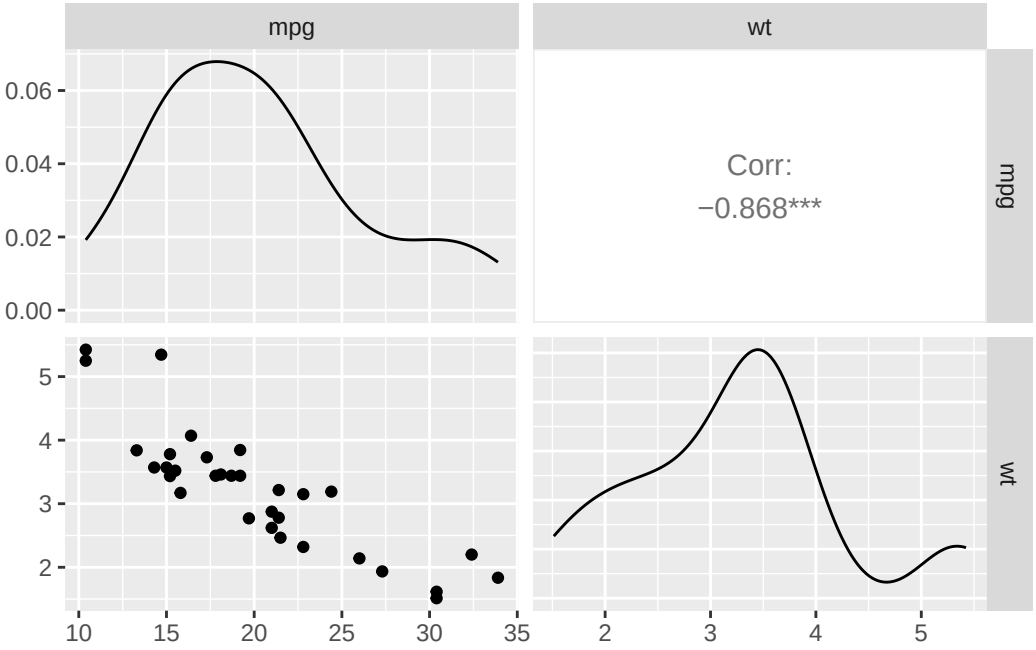
- Linearity
- Normal distribution of both variables (for Pearson)
- No extreme outliers

8.1.4 Visualization

```r
plot(mtcars$wt, mtcars$mpg)
abline(lm(mpg ~ wt, data = mtcars), col = "red")
```




```
GGally::ggpairs( mtcars, columns = c(1, 6))
```



8.2 T-Tests

8.2.1 Types of T-Tests

- One-sample
- Two-sample (independent)
- Paired

8.2.2 Real Dataset Example: `ToothGrowth`

```
library(datasets)
data(ToothGrowth)
head(ToothGrowth)
```

```
   len supp dose
1  4.2   VC  0.5
2 11.5   VC  0.5
3  7.3   VC  0.5
```

```
4  5.8   VC  0.5
5  6.4   VC  0.5
6 10.0   VC  0.5
```

```r
t.test(len ~ supp, data = ToothGrowth, var.equal = TRUE)
```

```
    Two Sample t-test

data:  len by supp
t = 1.9153, df = 58, p-value = 0.06039
alternative hypothesis: true difference in means between group OJ and group VC is not equal
95 percent confidence interval:
 -0.1670064  7.5670064
sample estimates:
mean in group OJ mean in group VC
        20.66333         16.96333
```

8.2.3 Assumptions

- Independent samples
- Normally distributed data
- Equal variances (for `var.equal = TRUE`)

8.2.4 Check Assumptions

```r
shapiro.test(ToothGrowth$len[ToothGrowth$supp == "OJ"])
```

```
    Shapiro-Wilk normality test

data:  ToothGrowth$len[ToothGrowth$supp == "OJ"]
W = 0.91784, p-value = 0.02359
```

```r
shapiro.test(ToothGrowth$len[ToothGrowth$supp == "VC"])
```

```
    Shapiro-Wilk normality test

data:  ToothGrowth$len[ToothGrowth$supp == "VC"]
W = 0.96567, p-value = 0.4284
```

```r
var.test(len ~ supp, data = ToothGrowth)
```

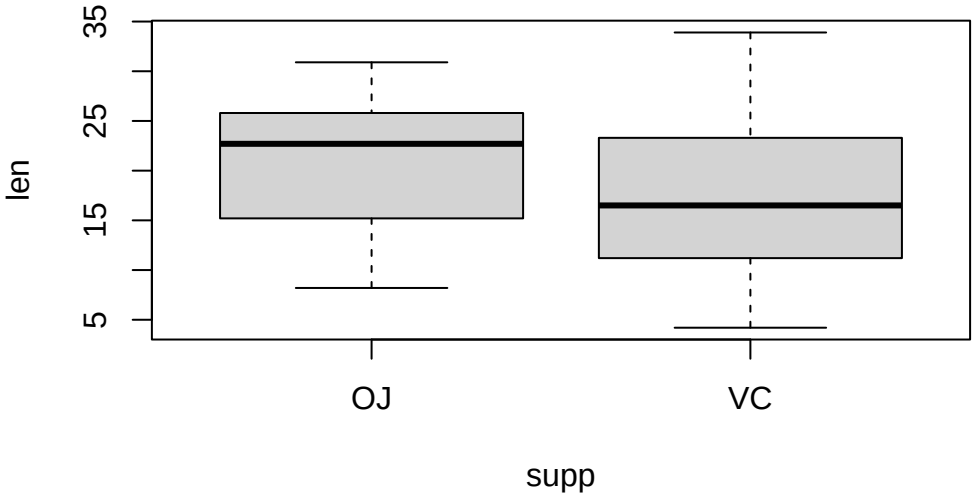
```
    F test to compare two variances

data:  len by supp
F = 0.6386, num df = 29, denom df = 29, p-value = 0.2331
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.3039488 1.3416857
sample estimates:
ratio of variances
```

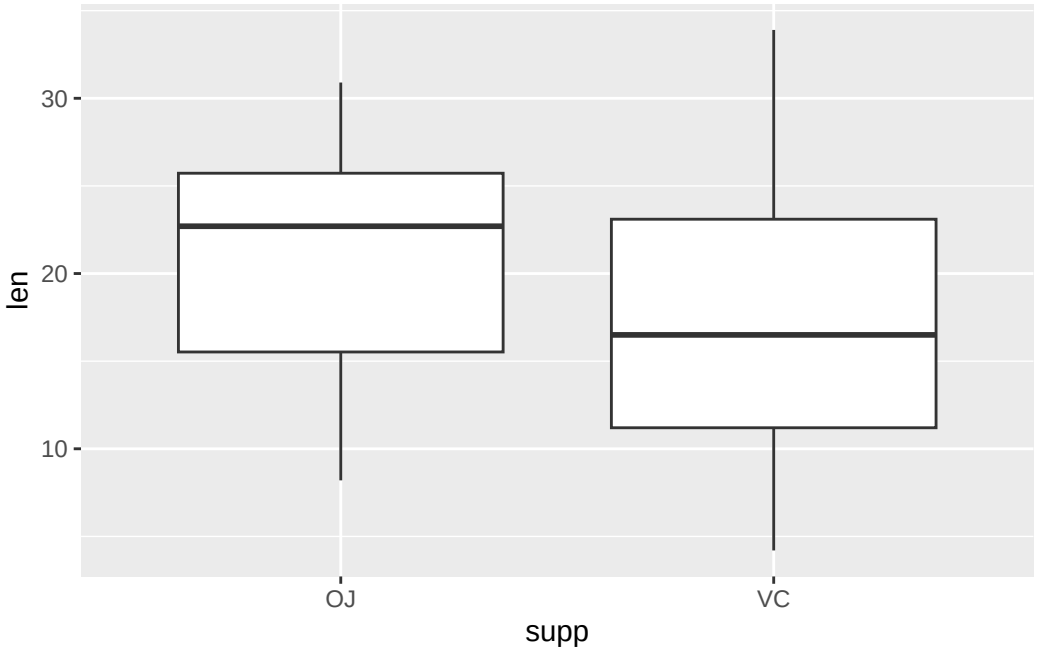
```
0.6385951
```

8.2.5 Visualization

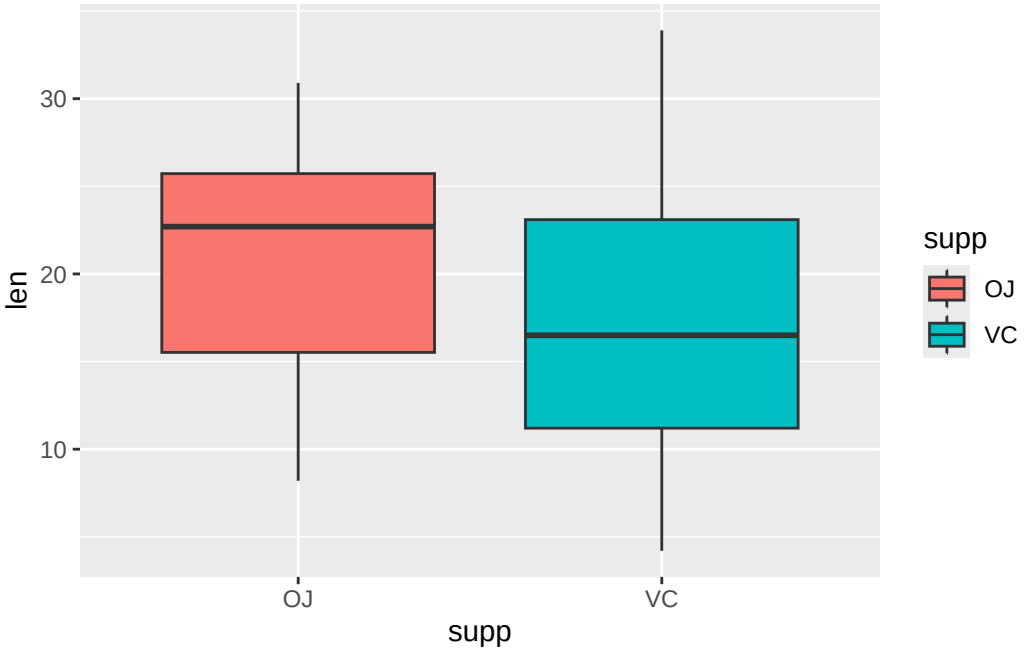
```
boxplot(len ~ supp, data = ToothGrowth)
```



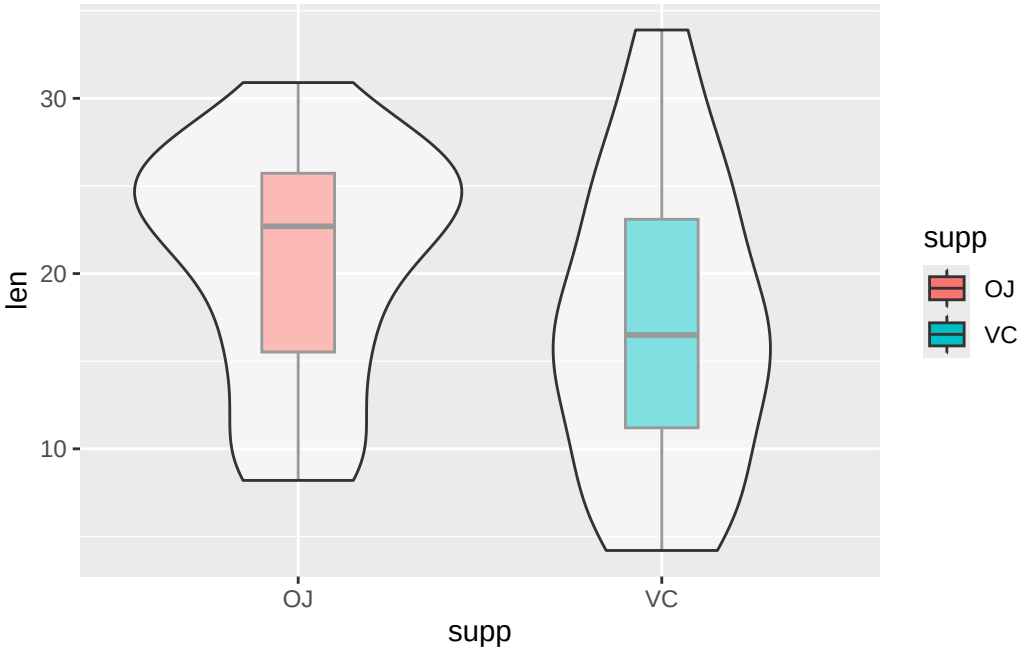

```
ggplot(ToothGrowth, aes(y = len, x = supp))+
  geom_boxplot()
```




```
ggplot(ToothGrowth, aes(y = len, x = supp))+
  geom_boxplot(aes(fill = supp))
```

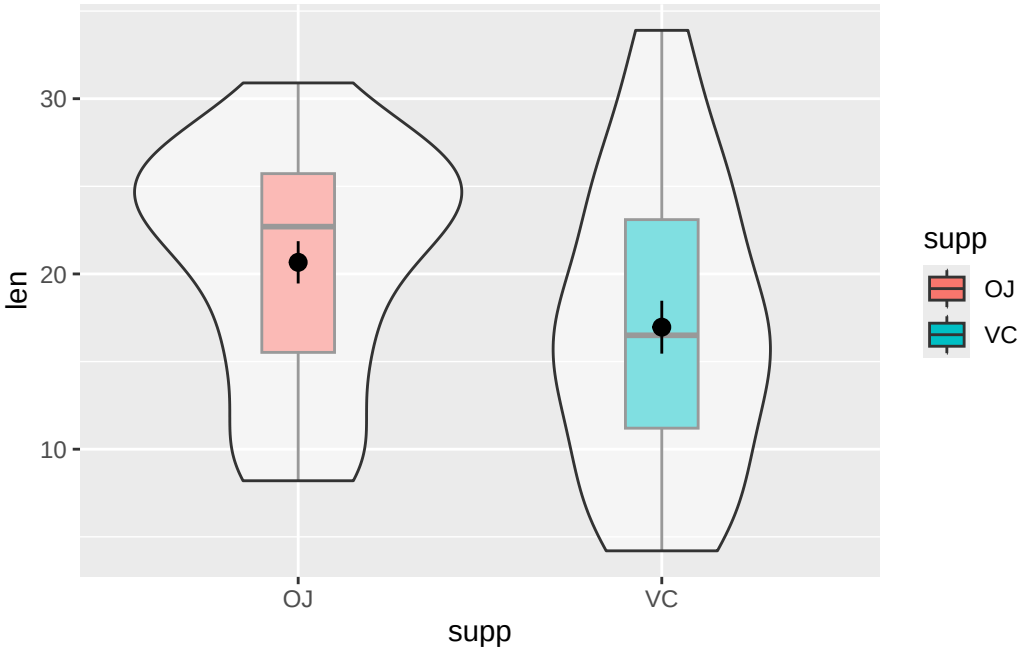



```
ggplot(ToothGrowth, aes(y = len, x = supp))+
  geom_boxplot(width=.2, aes(fill = supp))+
  geom_violin(alpha = 0.5)
```



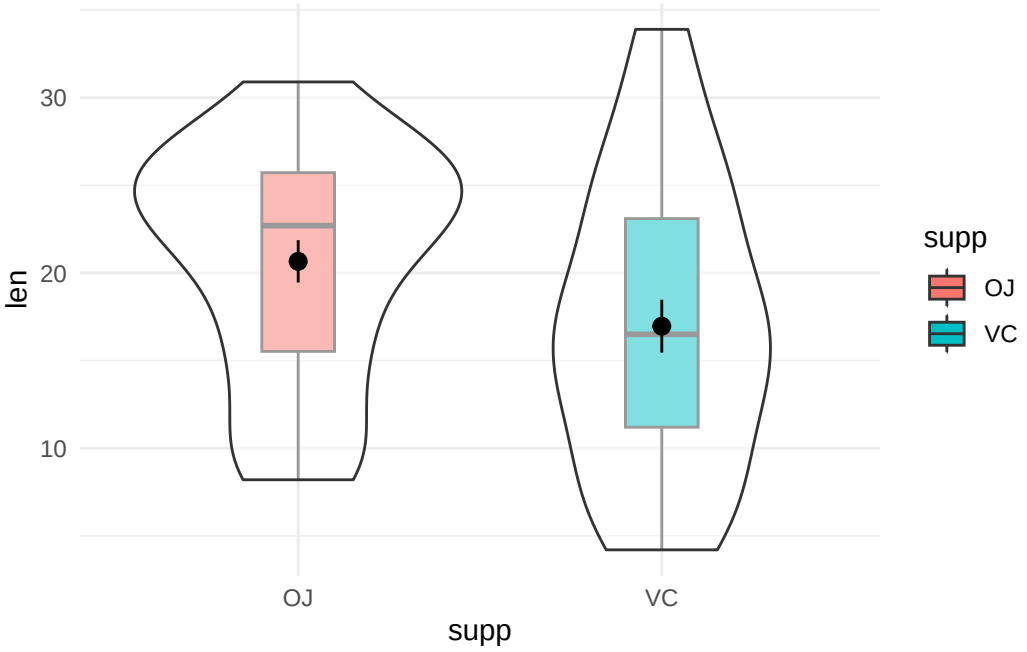

```
ggplot(ToothGrowth, aes(y = len, x = supp))+
  geom_boxplot(width=.2, aes(fill = supp))+
  geom_violin(alpha = 0.5)+
  stat_summary()
```

```
No summary function supplied, defaulting to `mean_se()`
```




```
ggplot(ToothGrowth, aes(y = len, x = supp))+
  geom_boxplot(width=.2, aes(fill = supp))+
  geom_violin(alpha = 0.5)+
  stat_summary() +
  theme_minimal()
```

```
No summary function supplied, defaulting to `mean_se()`
```



8.3 Wilcoxon Test

8.3.1 When to Use

- Non-parametric alternative to the t-test
- When assumptions of t-test are violated (e.g., non-normal data)

8.3.2 Real Dataset Example: `ToothGrowth`

```
wilcox.test(len ~ supp, data = ToothGrowth)
```

```
Warning in wilcox.test.default(x = DATA[[1L]], y = DATA[[2L]], ...): non è
possibile calcolare p-value esatto in presenza di ties
```

```

    Wilcoxon rank sum test with continuity correction

data:  len by supp
W = 575.5, p-value = 0.06449
alternative hypothesis: true location shift is not equal to 0
```

8.3.3 Assumptions

- Continuous or ordinal data
- Independent or paired depending on test
- Similar shape of distributions

8.4 Linear Models (LM)

8.4.1 What They Do

- Fit linear relationship between predictors and outcome
- `lm()` function in R

8.4.2 Real Dataset Example: `mtcars`

```
model <- lm(mpg ~ wt + hp, data = mtcars)
summary(model)
```

```

Call:
lm(formula = mpg ~ wt + hp, data = mtcars)

Residuals:
   Min     1Q Median     3Q    Max
-3.941 -1.600 -0.182  1.050  5.854

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.22727    1.59879  23.285  < 2e-16 ***
wt          -3.87783    0.63273  -6.129 1.12e-06 ***
hp          -0.03177    0.00903  -3.519  0.00145 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

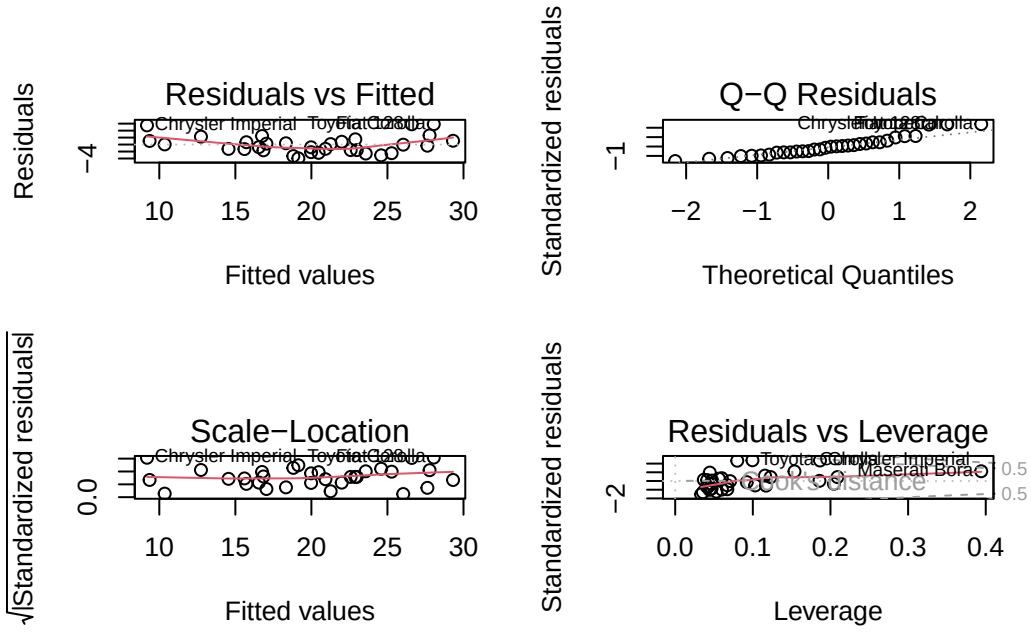
Residual standard error: 2.593 on 29 degrees of freedom
Multiple R-squared:  0.8268,    Adjusted R-squared:  0.8148
F-statistic: 69.21 on 2 and 29 DF,  p-value: 9.109e-12
```

8.4.3 Assumptions

- Linearity
- Homoscedasticity
- Normality of residuals
- Independence of errors

8.4.4 Diagnostic Plots

```
par(mfrow = c(2, 2))
plot(model)
```



8.5 Model Inference

8.5.1 Coefficients Interpretation

```
summary(model)
```

```
Call:
lm(formula = mpg ~ wt + hp, data = mtcars)

Residuals:
   Min     1Q Median     3Q    Max
-3.941 -1.600 -0.182  1.050  5.854

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.22727    1.59879  23.285  < 2e-16 ***
wt          -3.87783    0.63273  -6.129 1.12e-06 ***
hp          -0.03177    0.00903  -3.519  0.00145 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 2.593 on 29 degrees of freedom
Multiple R-squared:  0.8268,    Adjusted R-squared:  0.8148
F-statistic: 69.21 on 2 and 29 DF,  p-value: 9.109e-12
```

- p-values for hypothesis tests on coefficients
- R-squared for model fit

8.5.2 Confidence Intervals

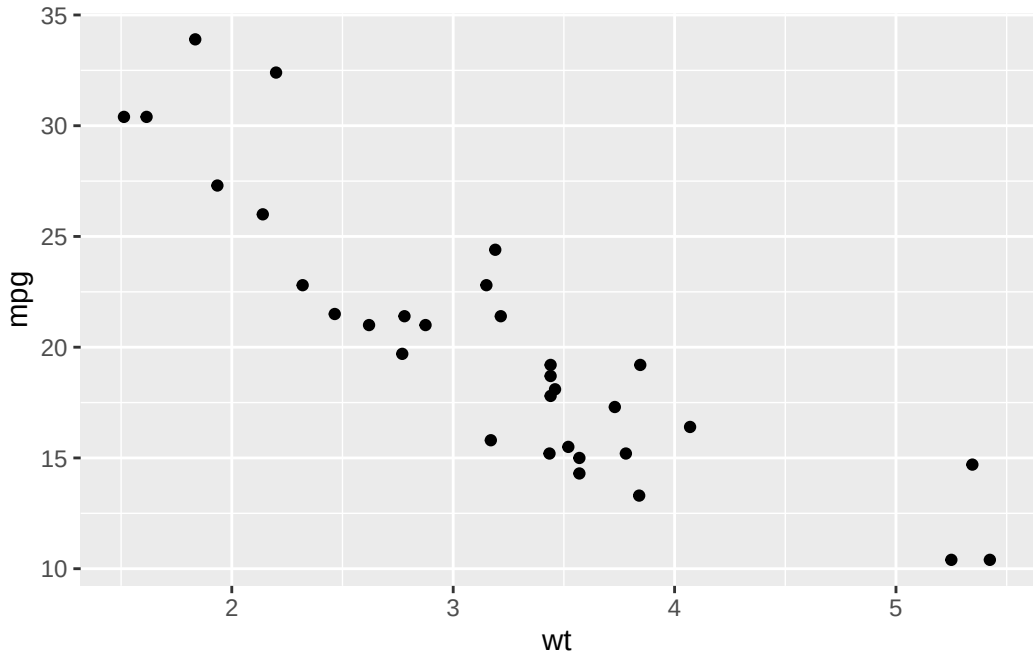
```
confint(model)
```

```
                2.5 %      97.5 %
(Intercept) 33.95738245 40.49715778
wt          -5.17191604 -2.58374544
hp          -0.05024078 -0.01330512
```

8.6 Visualization: Linear Model

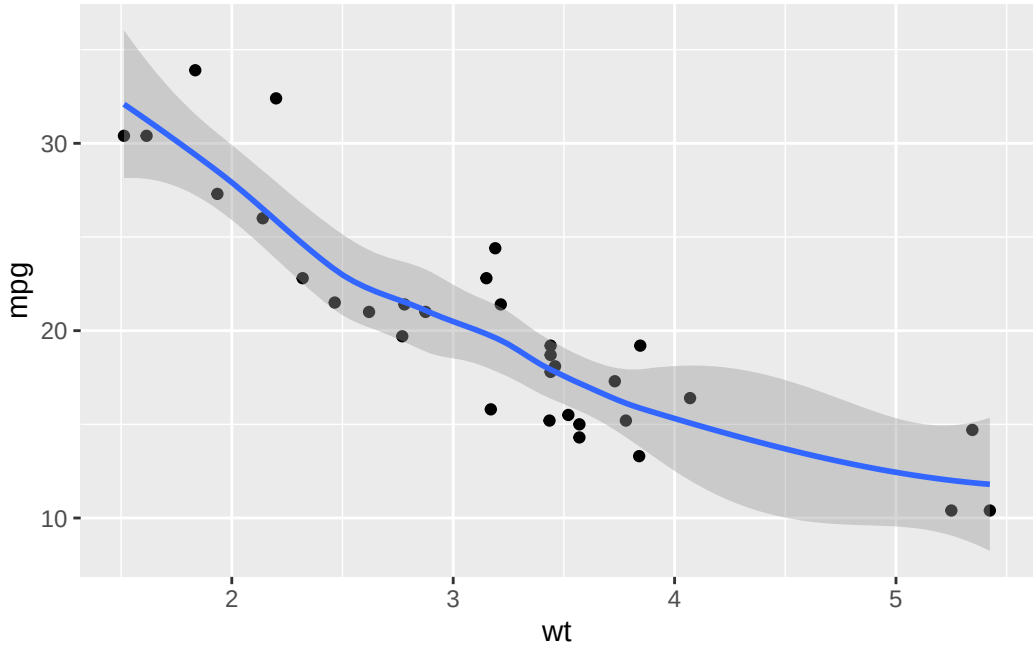
```
library(ggplot2)

ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```



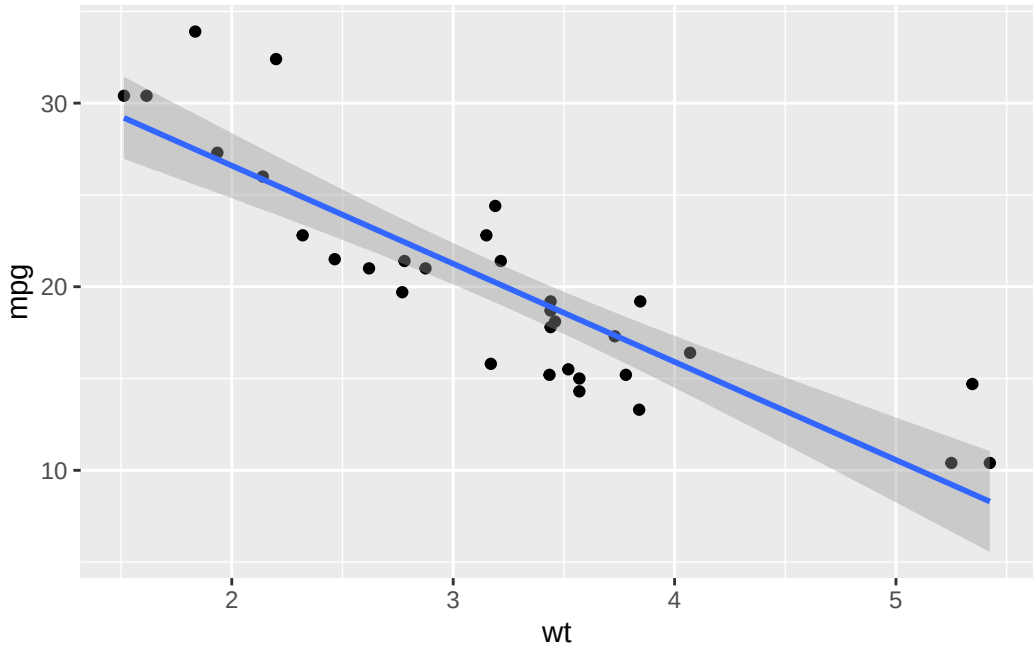

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



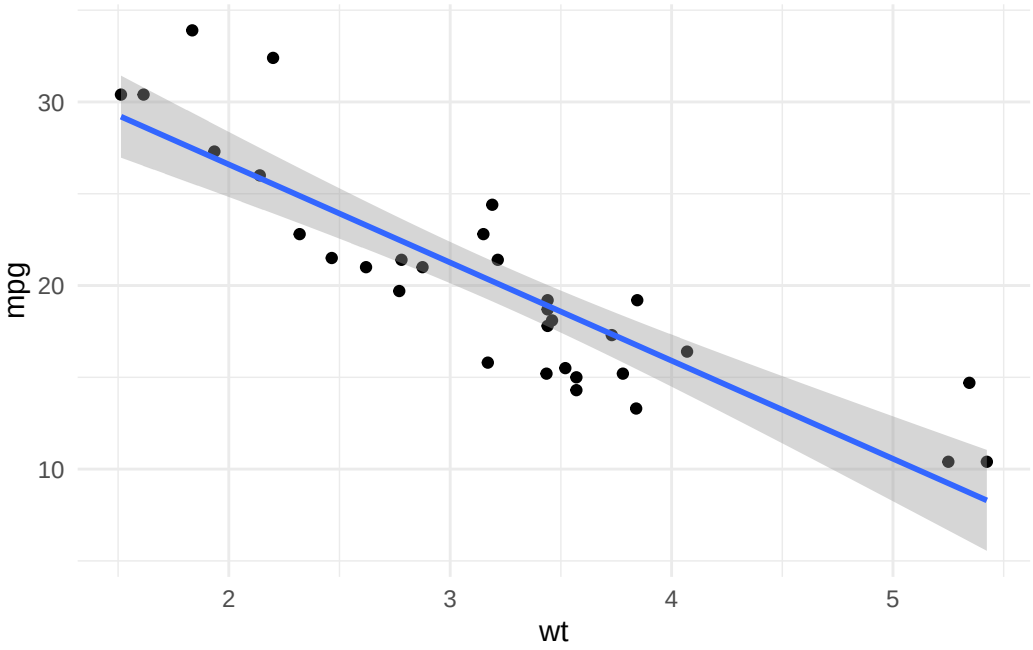

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth(method = "lm")
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE) +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```



8.7 Summary Table

Method	Assumptions	Inference Tools
Correlation	Linearity, normality (Pearson)	`cor.test()`
T-Test	Normality, equal variance	`t.test()`, `var.test()`
Wilcoxon	None strict (non-parametric)	`wilcox.test()`
Linear Model	Linearity, normality, homoscedasticity	`lm()`, `summary()`

9 RStudio Projects and the R Workspace

9.1 What is an RStudio Project?

- A **self-contained working directory** managed by RStudio
- Contains:
  - `.Rproj` file
  - Script files, data, outputs
- Promotes **reproducibility** and **organization**

9.2 Creating an RStudio Project

1. File > New Project
2. Choose:
   - New Directory
   - Existing Directory
   - Git Repository
3. A `.Rproj` file is created
4. Opens a clean R session in that folder

9.3 Benefits of Using Projects

- Isolates work in its own environment

- Keeps paths relative (avoid hard-coded paths)
- Better integration with Git
- Useful for collaboration and reproducibility

```
# bad
read.csv("C:/Users/me/Desktop/data.csv")

# good
read.csv("data/data.csv")
```

9.4 R Workspace: What's in It?

- Objects you create: variables, functions, models
- Stored in memory **until you quit R**
- Can be saved with `.RData` or `.RDS`

```
x <- 42
lm_model <- lm(mpg ~ wt, data = mtcars)
ls()  # Lists objects in workspace
```

9.5 Workspace Files

- `.RData`: Automatically saved workspace (discouraged in projects)
- `.Rhistory`: Command history
- You can choose whether to save the workspace on exit

```
save.image()      # Saves entire workspace
save(x, file = "x.RData")
saveRDS(model, "model.rds")
```

9.6 Best Practices

- Avoid saving workspace on exit (`.RData`)
- Start scripts with clean sessions
- Load only what you need
- Use projects to organize work
- Use `git` for Version control

9.7 Summary

Topic | Purpose |

||| | RStudio Project | Organize code/data in one directory | | R Workspace | Environment holding your R objects | | .RData/.RDS | Save and reload objects selectively |

10 Summary

- R is a statistical programming language widespread in Academia
- We analysed which are the main typologies of variables and functions
- We imported and manipulated datasets using R
- We performed some basic statistical analysis
- We visualised data with base and advanced R plots

10.1 Thank you for your attention!

My website:

Git-hub repository: